

České vysoké učení technické v Praze
Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství
Program: Aplikace informatiky v přírodních vědách
Specializace: –



Strojové učení pro adaptaci autonomních agentů

Machine learning for the adaptation of autonomous agents

VÝZKUMNÝ ÚKOL

Vypracoval: Bc. Martin Johec
Vedoucí práce: Ing. Josef Nový, Ph.D.
Rok: 2024

České vysoké učení technické v Praze

Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství

Akademický rok 2023/2024

ZADÁNÍ VÝZKUMNÉHO ÚKOLU

Student: Bc. Martin Jochec

Studijní program: Aplikace informatiky v přírodních vědách

Název práce: Strojové učení pro adaptaci autonomních agentů

Pokyny pro vypracování:

1. Nastudujte a popište problematiku umělé inteligence ve 3D akčních hrách.
2. Prozkoumejte možnosti strojového učení pro průběžnou adaptaci na chování protihráče.
3. Navrhněte algoritmy pro průběžnou adaptaci agenta na chování protihráče.
4. Navrhněte metodiky hodnocení úspěšnosti agenta.

Doporučená literatura:

- [1] Liébana, D. P., Lucas, S. M., Gaina, R. D., Togelius, J., Khalifa, A., & Liu, J. (2020). General Video Game Artificial Intelligence. In Synthesis Lectures on Games and Computational Intelligence. Springer International Publishing. <https://doi.org/10.1007/978-3-031-02122-0>
- [2] SUTTON, Richard S. a Andrew G. BARTO. Reinforcement Learning: An Introduction. Bradford Book, 1998. ISBN 0262039249.
- [3] ABBEEL, Pieter a John SCHULMAN. Continuous control with deep reinforcement learning. arXiv preprint, 2016. Dostupné z: <https://arxiv.org/abs/1509.02971>.
- [4] RUSSELL, Stuart a Peter NORVIG. Artificial Intelligence: A Modern Approach. Prentice Hall, 2020. ISBN 978-0136042594.

Jméno a pracoviště vedoucího práce:

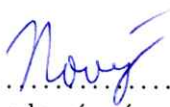
Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, ČVUT v Praze

Datum zadání výzkumného úkolu: 25. 10. 2023

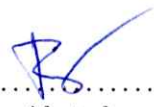
Termín odevzdání výzkumného úkolu: 25. 8. 2024

V Praze dne 25. 10. 2023

.....

vedoucí práce

.....

garant programu

.....

vedoucí katedry

Prohlášení

Prohlašuji, že jsem výzkumný úkol vypracoval samostatně a použil jsem pouze podklady (literaturu, projekty, SW atd.) uvedené v příloženém seznamu.

V Praze dne

.....
Bc. Martin Johec

Poděkování

Děkuji Ing. Josefovi Novému, Ph.D. za vedení mého výzkumného úkolu a pomoc při jeho tvorbě.

Bc. Martin Johec

Název práce:

Strojové učení pro adaptaci autonomních agentů

Autor: Bc. Martin Jocheč

Studijní program: Aplikace informatiky v přírodních vědách

Specializace: –

Druh práce: Výzkumný úkol

Vedoucí práce: Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, České vysoké učení technické v Praze

Konzultant: –

Abstrakt: Tento výzkumný úkol se zaměřuje na teorii a návrh kombinace umělé inteligence (AI) a strojového učení ve hrách za účelem dynamické úpravy chování herních protivníků v reálném čase. Úkol se skládá ze čtyř částí. První část pojednává o historii a typech umělé inteligence, konkrétně o umělé inteligenci v 3D akčních hrách. Druhá část zahrnuje základy strojového učení a jeho různých algoritmů. Třetí část navrhuje prostředí a algoritmy pro agenty AI v 3D akčních hrách a čtvrtá část nastiňuje metody vyhodnocování výkonu agentů, aby byla zajištěna co nejlepší efektivita agentů v dynamických herních prostředích.

Klíčová slova: autonomní agent, hry, umělá inteligence, adaptace, strojové učení

Title:

Machine learning for the adaptation of autonomous agents

Author: Bc. Martin Jocheč

Abstract: This research task focuses on the theory and design of combining Artificial Intelligence (AI) and machine learning in games to dynamically modify the behavior of game opponents in real time. The task consists of four parts. The first part discusses the history and types of AI, specifically AI in 3D action games. The second part covers the basics of machine learning and its various algorithms. The third part proposes environments and algorithms for an AI agent in 3D action games, and the fourth part outlines methods for evaluating agent performance to ensure the best agent effectiveness in dynamic game environments.

Key words: autonomous agent, games, artificial intelligence, adaptation, machine learning

Obsah

Úvod	9
1 AI ve hrách	11
1.1 Historie	11
1.2 „Chování pro tento případ“ neboli Ad-Hoc Behavior	14
1.2.1 Konečný automat (Finite State Machines)	14
1.2.2 Stromy chování (Behavior Trees)	16
1.2.3 „AI založená na užitečnosti“ neboli Utility System AI	17
1.3 Příklady dalších metod	19
1.3.1 Procházení stromu	19
1.3.2 Evoluční algoritmy	19
1.4 AI ve 3D akčních hrách	20
1.4.1 Obecné úkoly AI	20
1.4.2 F.E.A.R.	20
1.4.3 ME: Shadow of Mordor a ME: Shadow of War	22
2 Strojové učení (Machine Learning - ML)	25
2.1 Supervised Machine Learning	26
2.1.1 Klasifikace	27
2.1.2 Regrese	30
2.2 Unsupervised Machine Learning	31
2.2.1 Shlukování	31
2.2.2 „Těžba častých vzorů“ neboli asociace(FPM)	35
2.3 Semi-Supervised Machine Learning	39
2.3.1 Generativní adverzní síť (GAN)	39
2.4 Reinforcement Learning	41
3 Návrh autonomního agenta	45
3.1 Prostředí pro agenta	45
3.2 Algoritmy pro vytvoření agenta v akční hře	46
3.2.1 SARSA (State-Action-Reward-State-Action)	46
3.2.2 Deep Q-Learning	48
3.2.3 Rozdíly mezi těmito algoritmy	50
3.2.4 Použití obou algoritmů pro vytvoření agenta	50
4 Návrh hodnocení agenta	51
4.1 Metodika hodnocení	51

4.2	Typy hodnocení	53
4.2.1	Podle efektivnosti zabití nepřátel	53
4.2.2	Podle efektivnosti přežití	53
4.2.3	Efektivita zabití + přežití	53
Závěr		55
Literatura		56

Úvod

Tento výzkumný úkol se zabývá teorií a návrhem propojení umělé inteligence (AI) a strojového učení ve hrách k dynamickému přizpůsobování chování protivníků v reálném čase.

V první kapitole se popisuje umělá inteligence ve hrách. Specificky popisuje historii umělé inteligence ve stolních a digitálních hrách, dále rozebírá některé algoritmy, které jsou nejčastěji používány k vytvoření umělé inteligence v digitálních hrách, ale díky kterým není umělá inteligence schopna se přizpůsobit nebo upravit svoje chování. Nakonec se tato část specificky zaměří na některé existující 3D akční hry, které obsahují inteligentnější AI.

Ve druhé kapitole se zmiňují základy strojového učení a také existující čtyři typy strojového učení:

1. „Učení s učitelem“
2. „Učení bez učitele“
3. „Částečně řízené učení“
4. „Zpětnovazebné učení“

A také jeden nebo dva algoritmy z každého typu strojového učení.

Kapitola třetí obsahuje návrh autonomního agenta. Především obsahuje návrh samotného prostředí, kde se agent bude učit a zároveň testovat jeho inteligenci. Dále tato kapitola obsahuje dva algoritmy, SARSA a Deep Q-Learning, které budou použity pro sestavení agenta pro 3D akční hru.

Ve čtvrté kapitole řeším typy hodnocení, podle kterých by se agent rozhodoval, v čem se zlepšit a co nepotřebuje využívat. Jsou tu tři návrhy hodnocení, které by představovali zajímavou obtížnost pro agenta, aby se jim přizpůsobil.

Závěr obsahuje konečné myšlenky autora a také pokračování pro další práci, kde by se myšlenka tohoto výzkumného úkolu uskutečnila.

Kapitola 1

AI ve hrách

1.1 Historie

Prolínání her a umělé inteligence (AI) je dlouhodobě zkoumanou oblastí. Velká pozornost ve výzkumu AI týkající se her se zaměřuje na vývoj agentů schopných zapojit se do hry, ať už s integrací strojového učení, nebo bez ní. Tato cesta zkoumání historicky představovala primární a často jediný přístup k využití umělé inteligence v herním kontextu.

Ještě před formalizací umělé inteligence jako samostatného oboru se průkopníci počítačové vědy snažili prozkoumat možnosti výpočetní techniky tím, že navrhovali programy schopné hrát stolní hry. Mezi těmito prvními snahami je práce Alana Turinga, který je obecně považován za zásadní postavu počítačové vědy a který upravil algoritmus Minimax pro hraní šachů.

Počátky zapojení umělé inteligence do digitálních her lze vysledovat až k průkopnické práci A.S. Douglase z roku 1952, kdy v rámci své doktorské práce na univerzitě v Cambridge vyvinul software schopný naučit se dokonale hrát Tic-Tac-Toe. Příspěvky Arthura Samuela následně znamenaly významný milník v podobě zavedení takzvaného „reinforcement“ učení, předchůdce současných technik strojového učení, demonstrovaného na programu, který autonomně zlepšoval svojí taktiku v dámě prostřednictvím opakovaného samohraní[1].

Raný výzkum v oblasti umělé inteligence zaměřené na hraní her se převážně soustředil na tradiční deskové hry, jako je dáma nebo šachy. Tyto hry byly obzvláště zajímavé díky své schopnosti generovat složité komplexy z jednoduchých souborů pravidel a díky svému historickému postavení po staletí testovala hranice lidského poznání. V roce 1994 došlo k významnému průlomů, když program Chinook Checkers porazil úřadujícího mistra světa v dámě Mariona Tinsleyho. Následně v roce 2007 byla dáma zcela vyřešena[2], což znamenalo významný milník ve výzkumu AI.

Šachy, často označované jako „octomilka umělé inteligence“, posloužily jako klíčové měřítko pro testování nových metod umělé inteligence. Vývoj softwaru, který je schopen překonat lidský výkon v šachu, však posunul zaměření výzkumu AI na jiné oblasti. Deep Blue[4] od společnosti IBM, využívající algoritmus Minimax rozšířený o úpravy specifické pro šachy a vysoce optimalizovanou funkci vyhodnocování šachovnic, v roce 1997 slavně porazil mistra světa Garryho Kasparova.

Dalším významným pokrokem, který předcházel úspěchům Deep Blue a Chinook, je demonstrován softwarem TD-Gammon[3], který v roce 1992 vyvinul Gerald Tesauro. TD-Gammon se vyznačuje využitím umělé neuronové sítě vycvičené pomocí „temporal difference learning“, která se sama proti sobě opakovaně zapojuje do hry. Pozoruhodné je, že TD-Gammon dosáhl dovedností, které jsou srovnatelné s uznávanými lidskými hráči vrhcábu, což dokazuje účinnost jeho tréninkové metodiky.

Po průlomových objevech v oblasti umělé inteligence v tradičních deskových hrách dosáhla vrcholu v oblasti umělé inteligence v deskových hrách hra Go v roce 2016. Hra Go se stala novou hračkou pro herní umělou inteligenci, která se vyznačuje bezkonkurenčním faktorem větvení blížícím se k číslu 250 a rozsáhlým prohledávacím prostorem, který řádově převyšuje prostor šachů. Ze začátku bylo dosažení lidské úrovně v Go považováno za vzdálenou budoucnost, ale nakonec v říjnu 2015 odehrál AlphaGo, který patří společnosti Google DeepMind a je vybaven revolučním frameworkem „Deep reinforcement learning“, svůj první zápas proti úřadujícímu trojnásobnému mistru Evropy Fan Hui, který i vyhrál a skóroval 5:0. V roce 2016 Lee Sedol, profesionální hráč Go s devíti tituly, podlehl porážce v zápase na pět her proti AlphaGo. Díky těmto drtivým vítězstvím se hra Go stala posledních stolní hrou, kde počítače dosáhly nadlidských schopností v klasických deskových hrách, protože další stolní hry zatím lidské hráče v širokém měřítku nezaujaly.[5]

Tradiční deskové hry, které se vyznačují strukturovaným tahovým hraním a transparentním sdílením informací mezi hráči, však představují pouze podmnožinu herních prostředí. Uznává se, že inteligence zahrnuje širší dimenze než ty, které jsou testovány v těchto tradičních hrách.

V posledních patnácti letech se vytvořila rostoucí komunita, která se zaměřuje především na zlepšování schopností umělé inteligence v digitálních hrách s cílem dosáhnout výkonnosti srovnatelné s lidskými hráči, replikovat specifické lidské strategie nebo vykazovat jedinečné vlastnosti.

K významnému pokroku v této oblasti došlo v roce 2014, kdy algoritmy vyvinuté společností Google DeepMind prokázaly schopnost vyniknout v různých hrách z klasické konzole Atari 2600 a překonaly výkon na úrovni člověka pouze za použití surových pixelových vstupů. Mezi těmito hrami představovala výzvu pro konvenční přístupy hra Ms. Pac-Man (Namco, 1982). Tým Microsoft Maluuba však tuto výzvu vyřešil použitím techniky posilovacího učení s hybridní architekturou odměn, čímž nakonec dosáhl řešení této hry.

V poslední době však došlo k prudkému nárůstu výzkumu zaměřeného na vývoj věrohodných agentů ve hrách, což otevřelo cestu novým směrům v herní umělé inteligenci.

Jednou z cest zkoumání je vytváření agentů schopných projít herním testem, který se jmenuje Turing test[6]. Tento test, odvozený od klasického Turingova testu, vyžaduje, aby porota přesně rozeznala, zda pozorované herní chování pochází od lidského hráče nebo od herního bota řízeného umělou inteligencí. V roce 2012 se dva boty řízené umělou inteligencí ve hře Unreal Tournament 2004 od Epic Games úspěšně prokázaly schopnost projít herním Turingovým testem, což svědčí o jejich schopnosti replikovat herní chování podobné lidskému.

1.2 „Chování pro tento případ“ neboli Ad-Hoc Behavior

V oblasti vývoje her je již dlouho známé tzv. „Ad-Hoc Behavior“, jako jsou konečné stavové stroje, stromy chování a umělá inteligence založená na užitku. Tyto metody tradičně umožňují značnou kontrolu nad nehráčskými postavami (NPC). O jejich trvalé převaze svědčí i to, že se s jejich používáním v komunitě vývojářů her stále spojuje termín „herní umělá inteligence“.

1.2.1 Konečný automat (Finite State Machines)

Až do poloviny 20. století byl dominantní metodou pro ovládání a rozhodování nehráčských postav (NPC) ve hrách konečný automat (Finite State Machines, KA) [7] a jeho varianty, například hierarchické KA. Tyto techniky měly v oblasti herní umělé inteligence převahu a prokázaly svou účinnost při řízení chování NPC v herním prostředí.

Konečné automaty spadají do oblasti expertních znalostních systémů a běžně se vizualizují jako grafy. Tyto grafy slouží jako abstraktní reprezentace vzájemně propojených prvků, jako jsou objekty, symboly, události, akce nebo vlastnosti relevantní pro konkrétní kontext návrhu. V rámci této reprezentace uzly odpovídají stavům, z nichž každý je charakterizován matematickou abstrakcí, zatímco hrany označují přechody, které znamenají podmíněné vztahy mezi stavy. KA fungují tak, že v daném okamžiku může být aktivní pouze jeden stav, přičemž k přechodům dochází na základě splnění předem definovaných podmínek. V podstatě jsou charakterizovány třemi základními prvky:

1. Stav:

- Obsahují informace týkající se úlohy, jako je například aktuální stav „zkoumání“.

2. Přechody:

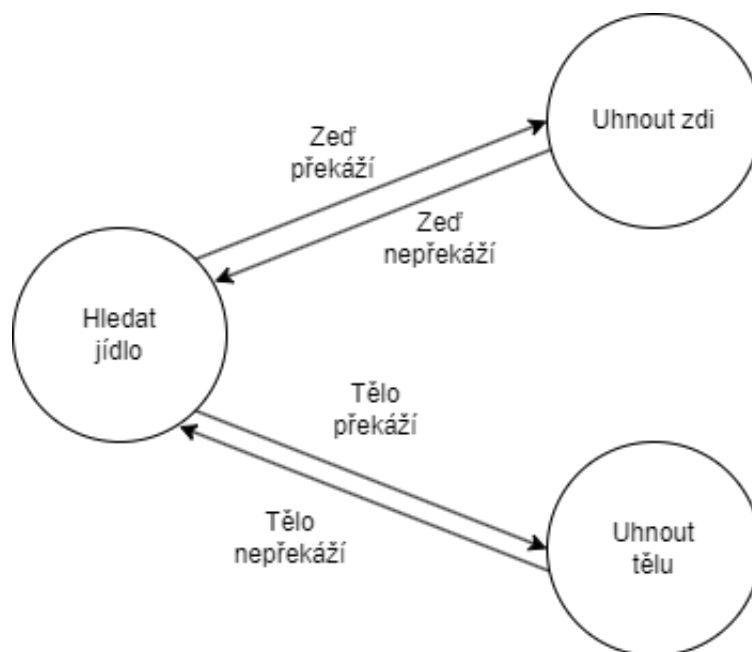
- Označují změny stavu vyvolané podmínkami, které musí být splněny, jako je přechod do stavu „upozorněn“ po zaslechnutí střelby.

3. Akce:

- Určují specifické chování, jež má být provedeno, například náhodný pohyb a hledání protivníka ve stavu „průzkum“.

Konečné automaty nabízejí pozoruhodné výhody díky své jednoduchosti při návrhu, implementaci, vizualizaci a ladění. Při aplikaci na rozsáhlejší systémy však mohou představovat výzvu kvůli své potenciální složitosti, což vede k výpočetním omezením v některých aspektech.

Kromě toho mají významnou nevýhodou, spolu s dalšími ad-hoc behavior metodami, a to je jejich přirozená nedostatečná flexibilita a dynamika, pokud nejsou speciálně navrženy tak, aby tyto vlastnosti zahrnovaly. Jakmile jsou dokončeny, otestovány a odladěny, jejich přizpůsobivost a evoluční kapacita jsou omezeny, což vede k projektům relativně předvídatelného chování v herním kontextu.



Obrázek 1.1: Příklad KA pro jednoduchou hru Snake. Obsahuje tři stavy a čtyři přechody. Had se postupně přibližuje k jídlu, ale pokud má před sebou zeď a nebo svoje tělo, tak se těmito prvky pokusí vyhnout.

1.2.2 Stromy chování (Behavior Trees)

Strom chování (Behavior Tree, SC)[8] funguje jako expertní znalostní systém, podobně jako konečný automat (Finite State Machine, KA), který usnadňuje přechody mezi konečnou množinou úloh nebo chování. Významná výhoda SC oproti KA spočívá v jejich modulární struktuře, která umožňuje skládání složitých chování z jednodušších úloh. Na rozdíl od KA, které představují především stavy, jsou SC strukturovány kolem chování, čímž se odlišují od hierarchických KA. Podobně jako KA nabízejí SC snadný návrh, testování a ladění, což přispělo k jejich širokému rozšíření při vývoji her po úspěšných implementacích v titulech, jako je Halo 2[9] a Bioshock.

Využívají hierarchickou stromovou strukturu, která se skládá z kořenového uzlu, nadřazených uzlů a jim odpovídajících podřízených uzlů, z nichž každý představuje specifické chování (viz obrázek 1.2). Procházení SC začíná od kořene a aktivuje provádění dvojic rodič-dítě, jak je ve stromu naznačeno. Během předem stanovených časových kroků může podřízený uzel vrátit svému rodiči jednu z následujících hodnot: „běží“, pokud chování zůstává aktivní, „úspěch“ při úspěšném dokončení nebo „selhání“, pokud chování selže. SC se skládají ze tří základních typů uzlů: sekvence, selektor a dekorátor, z nichž každý plní odlišné funkce, které jsou podrobněji popsány níže:

1. Sekvence (na obr.1.2 znázorněna šedou barvou):

- Úspěch v podřízeném chování vyvolá pokračování v sekvenci, což nakonec vede k úspěchu nadřazeného uzlu, pokud všechna podřízená chování uspějí; naopak, sekvence selže, pokud některé podřízené chování selže.

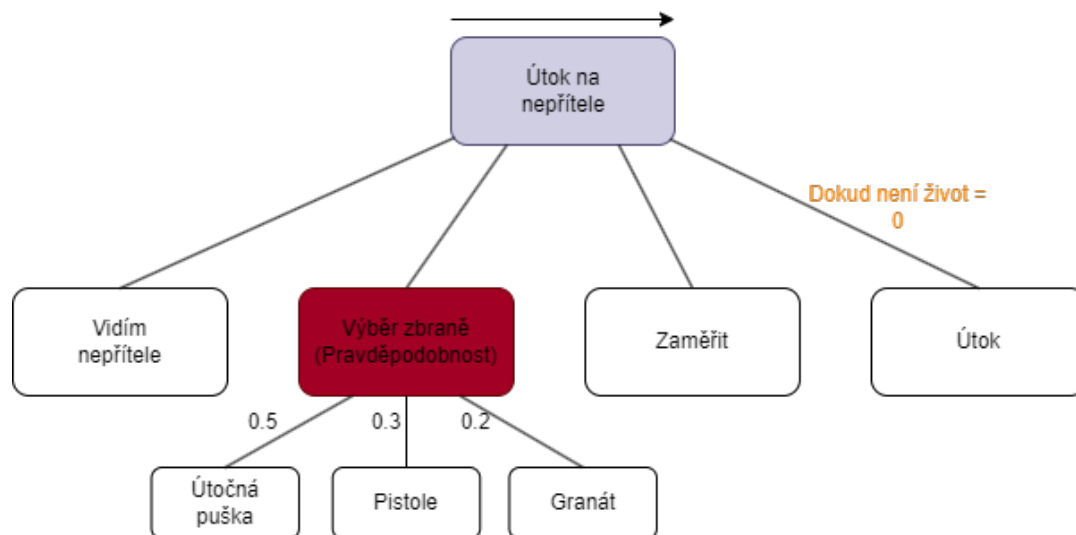
2. Selektor (na obr.1.2 znázorněn červenou barvou):

- Uzly selektorů zahrnují dva základní typy: selektory pravděpodobnosti a selektory priority. U pravděpodobnostních selektorů jsou podřízená chování vybírána na základě pravděpodobností předem stanovených tvůrcem SC. Naopak prioritní selektory řadí podřízená chování do seznamu a zkoušejí je postupně. Bez ohledu na typ selektoru vede úspěch v podřízeném chování k úspěchu selektoru. Při neúspěchu v podřízeném chování je vybráno další podřízené chování v pořadí (u prioritních selektorů) nebo selže samotný selektor (u pravděpodobnostních selektorů).

3. Dekorátor (na obr.1.2 znázorněn oranžovou barvou):

- Uzly dekorátoru rozšiřují složitost a možnosti jednotlivých podřízených chování. Mezi příklady dekorátorů patří určení počtu iterací, které podřízené chování podstoupí, nebo přidělení časového limitu pro jeho dokončení.

Ve srovnání s KA nabízejí větší flexibilitu při navrhování a jsou relativně jednodušší na testování. Nicméně mají podobná omezení, zejména pokud jde o jejich statickou povahu, která omezuje jejich přizpůsobivost a dynamičnost. Zatímco pravděpodobnostní výběrové uzly vnášejí prvek nepředvídatelnosti, jsou snahy o zvýšení adaptability prostřednictvím úprav ve struktuře stromů.



Obrázek 1.2: Příklad stromu chování

1.2.3 „AI založená na užitečnosti“ neboli Utility System AI

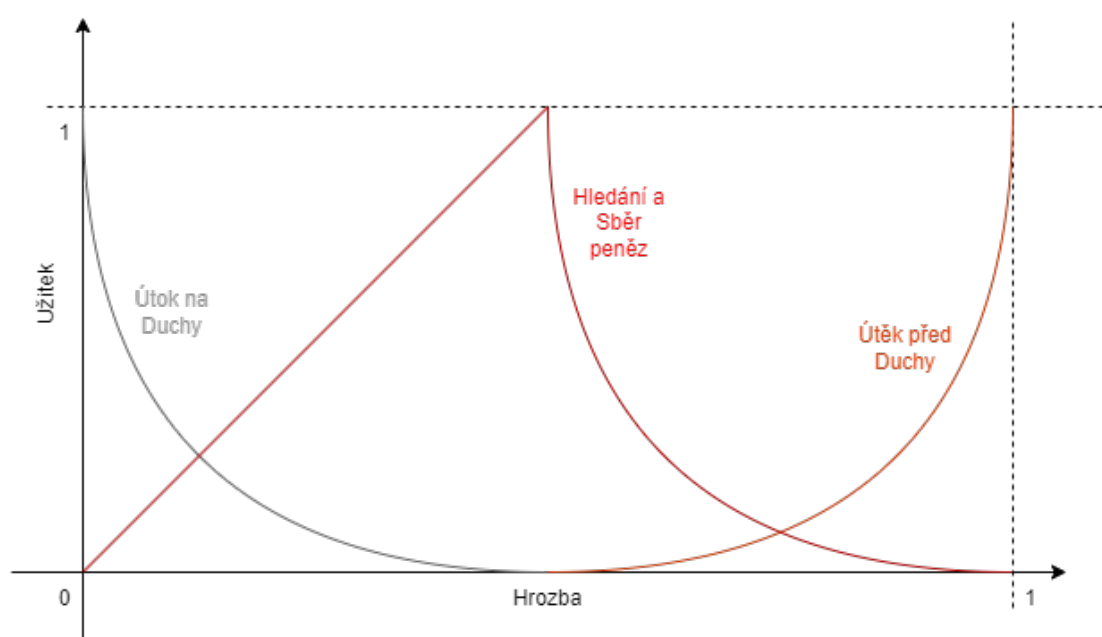
V odvětví herní umělé inteligence je velká absence modularity chování v různých hrách a představuje významnou překážku pro dosažení vysoce kvalitní umělé inteligence. K překonání omezení KA a SC v tomto ohledu je stále oblíbenějším přístupem umělá inteligence založená na užitečnosti (Utility System AI, USAI)[10]. Tato metoda nabízí prostředky pro efektivnější návrh řídicích a rozhodovacích systémů ve hrách. V USAI je ke každé herní instanci přiřazena funkce užitku, která kvantifikuje její důležitost v kontextu hry. Tyto funkce mohou například hodnotit faktory, jako je blízkost nepřítele nebo zdravotní stav agenta. Na základě zvážení všech dostupných užitečných hodnot a potenciálních akcí určí umělá inteligence nejzásadnější možnost, kterou je třeba v daném okamžiku upřednostnit. Tento přístup vychází z principů teorie užitku z ekonomie a zahrnuje návrh funkcí užitku, podobně jako vytváření funkcí příslušnosti v teorii fuzzy množin.

Užitky zahrnují široké spektrum ukazatelů, od objektivních údajů, jako je zdraví nepřítele, až po subjektivní faktory, jako jsou emoce a vnímané hrozby. Tyto různorodé metriky spojené s potenciálními akcemi nebo rozhodnutími lze sloučit do lineárních nebo nelineárních vzorců, které slouží k vedení agentů při rozhodování na základě souhrnného skóre užitečnosti. Na rozdíl od sekvenčních rozhodovacích procesů KA a SC umožňuje USAI současné vyhodnocení všech dostupných možností. Přiřazením užitku každé možnosti a výběrem té s nejvyšším užitekem mohou agenti určit nejvhodnější postup.

Tyto hodnoty užítu podléhají pravidelnému přehodnocování, například přepočty probíhající v předem stanovených intervalech během hry, aby se zajistilo sladění s dynamickým stavem hry.

USAI má oproti jiným technikám ad-hoc chování výrazné výhody. Její modularita umožňuje herním agentům rozhodovat se na základě dynamického souboru faktorů, což podporuje adaptabilitu rozhodovacích procesů. Rozšiřitelnost navíc umožňuje bezproblémovou integraci nových úvah podle potřeby.

Opakovaná použitelnost metody navíc usnadňuje přenos užitekových komponent mezi rozhodnutími a napříč různými hrami, což zvyšuje efektivitu vývoje. Tento přístup se hojně využívá v různých herních žánrech, přičemž jeho významné implementace byly zaznamenány ve hrách, jako je Red Dead Redemption pro výběr zbraní a dialogů, a také ve hře F.E.A.R. pro dynamické taktické rozhodování[11].



Obrázek 1.3: Řízení chování hry Pac-Man založené na USAI. Hrozba (osa x) je funkce, která leží mezi 0 a 1 a je založena na aktuální pozici duchů. Pac-Man bere v úvahu aktuální úroveň ohrožení, přiřazuje hodnoty užítu (prostřednictvím tří různých křivek) a rozhodne se sledovat chování s nejvyšší hodnotou užítu. V tomto příkladu roste exponenciálně útěk před duchy s ohledem na ohrožení. Naopak útok na duchy exponenciálně klesá s ohledem na ohrožení. Nakonec hledání a sběr peněz roste lineárně až do určité úrovně ohrožení, od tohoto bodu klesá exponenciálně.

1.3 Příklady dalších metod

1.3.1 Procházení stromu

Procházení stromem[12] je základní úlohou v teorii algoritmizace a zahrnuje systematické zkoumání všech uzlů v rámci datové stromové struktury. Vzhledem k analogii stromu s grafem je procházení stromů úzce spjato s teorií grafů. Metody procházení stromů se běžně demonstrují na binárních stromech, ale lze je snadno rozšířit i na jiné stromové struktury. Dva základní přístupy, prohlédávání do hloubky (DFS) a prohlédávání do šířky (BFS), se liší pořadím návštěv uzlů a použitím pomocných datových struktur. DFS, často rekurzivní, obvykle využívá zásobník, a to buď explicitně, nebo implicitně prostřednictvím použití zásobníku volání. Naproti tomu BFS typicky využívá frontu.

Ve scénářích, kde je cílem cílené vyhledávání hodnot spíše než úplné procházení stromu, umožňují specializované konstrukční techniky, jako je vytváření vyhledávacího stromu (typicky binárního vyhledávacího stromu), efektivní použití přizpůsobených variant algoritmů. Další specializované algoritmy jsou použitelné v problémech prohlédávání stavového prostoru, které lze reprezentovat jako stromy. Pozoruhodné je, že heuristické algoritmy, včetně stromového vyhledávání Monte Carlo odvozeného z metody Monte Carlo, našly úspěch v aplikacích umělé inteligence v rámci počítačových her.

1.3.2 Evoluční algoritmy

Evoluční algoritmy (EA)[13] jsou podmnožinou evolučních výpočtů a představují metaheuristické optimalizační algoritmy založené na populaci. Tyto algoritmy čerpají inspiraci z biologické evoluce a využívají mechanismy, jako je reprodukce, mutace, rekombinace a selekce. V EA představují kandidáti na řešení optimalizačních problémů jedince v populaci, jejichž kvalita se hodnotí pomocí fitness funkce. Iterativním použitím těchto operátorů se populace vyvíjí. Výpočetní složitost, zejména spojená s vyhodnocováním fitness funkcí, často představuje v reálných implementacích EA značnou výzvu. K řešení tohoto problému se používají metody, jako je aproximace fitness. Navzdory své zdánlivé jednoduchosti vykazují EA účinnost při řešení složitých problémů, což naznačuje, že složitost algoritmu nemusí vždy přímo korelovat se složitostí problému.

1.4 AI ve 3D akčních hrách

1.4.1 Obecné úkoly AI

V 3D akčních hrách by se měla umělá inteligence umět pohybovat virtuálním světem, takže například ovládat základní pohyby po mapě ale také pokročilé strategie jako například vylézt po zdi nebo skrčit se a projít ventilační šachtou a tak dále. Klíčové jsou také poznávací schopnosti a přizpůsobivost, neboli například dokázat předvídat hráčovi interakce s objekty nebo se samotnou umělou inteligencí. AI by mělo také vynikat v boji a používat různé taktiky, jako je obcházení hráče a nebo koordinované útoky. Kromě boje by měli zvládat i úkoly, jako je interakce s prostředím nebo pomáhat hráčovi dosáhnout cíle. Pokud hra obsahuje i příběhovou stránku, tak by měla AI umět mimikovat emoce, aby dokázala vyjadřovat řadu pocitů, aby hráč se vcítil do své postavy a tím pádem do herního světa.

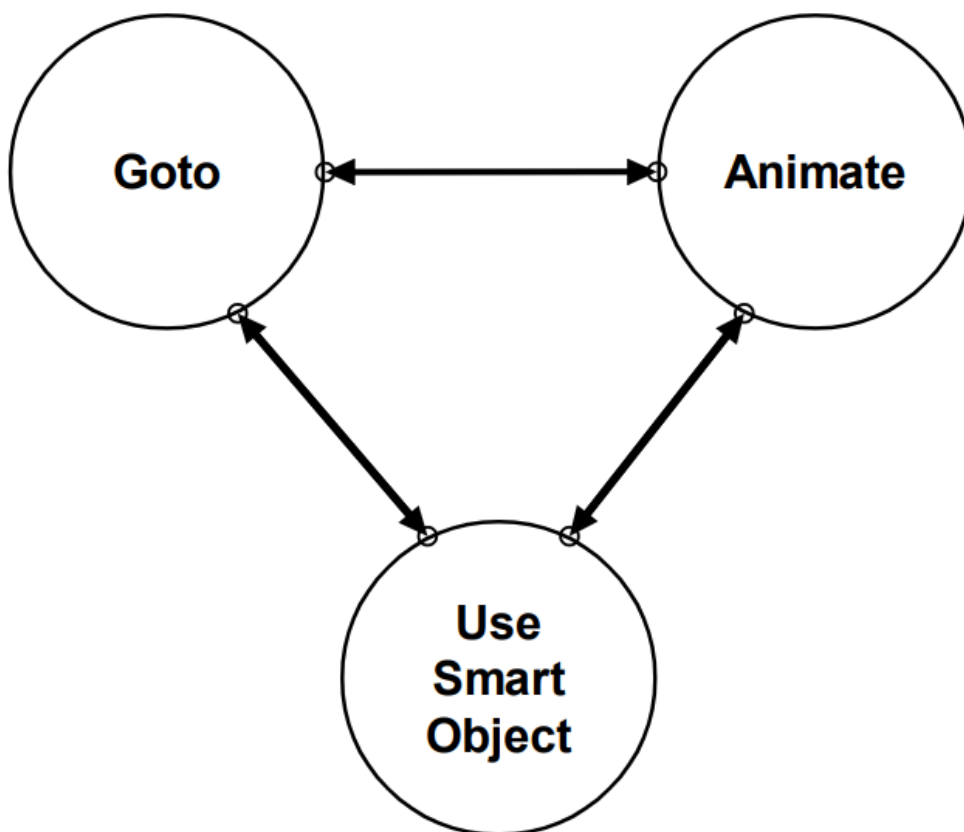
1.4.2 F.E.A.R.

Hra F.E.A.R.[14][15], zkratka pro First Encounter Assault Recon, debutovala v roce 2005 jako střílečka z pohledu první osoby vyvinutá společností Monolith Productions. Navzdory svému stáří je stále chválena jako ukázkový příklad implementace umělé inteligence v bojových videohrách. Klíč k jejímu trvalému úspěchu spočívá v kombinaci inovativních technik umělé inteligence a strategického herního designu.

Jako první do hry implementovali techniku umělé inteligence zvanou automatizované plánování. Tento přístup, inspirovaný systémem STRIPS[16] vyvinutým v roce 1971 pro programování robotů, umožňuje nehráčským postavám (NPC) zapojit se do sofistikovaných rozhodovacích procesů v herním prostředí. Konkrétně jsou NPC vybaveny schopností plánovat svůj pohyb, vybírat krytí a rozhodovat o bojových akcích, jako je střelba nebo použití granátu. Tato integrace automatizovaného plánování zahrnuje přiřazení konkrétních cílů každé NPC a modelování možných akcí, které mají v herním světě k dispozici. Analýzou těchto možností mohou NPC vytvářet optimalizované plány pro efektivní dosažení svých cílů.

Každá akce umělé inteligence ve F.E.A.R. se řídí předpoklady a účinky. Předpoklady představují podmínky, které jsou ve světě hry nezbytné pro provedení akce, zatímco účinky označují výsledky vyplývající z provedení akce. Například, pokud je postava pozorována, jak běží k objektu a sráží jej na zem, aby jej použila jako úkryt, znamená to sekvenci plánovaných akcí: pohyb, manipulace s objektem a nakonec hledání úkrytu. To znamená, že před zahájením sekvence akcí je třeba zajistit, aby byl objekt vzprámený, dosažitelný a schopný poskytnout úkryt. Dodržování předpokladů zabráňuje nelogickým akcím, jako je pokus o manipulaci s objektem, který je již převrácený, nebo pohyb k nedosažitelnému objektu, čímž se zvyšuje realističnost NPC ve hře.

Kromě plánovacích funkcí využívá systém F.E.A.R. ke koordinaci provádění formulovaných strategií konečný automat (KA). Programátoři v Monolithu si uvědomily, že mohou mapovat každou akci umělé inteligence ve hře na odpovídající animaci, ať už jde o střelbu ze zbraně, přemísťování nebo interakci s prvky prostředí. Díky tomuto se zjednodušuje složitost chování AI tím, že v rámci KA rozděluje akce do tří různých stavů: pohyb na určené pozice, animační sekvence a specializované interakce s určenými „inteligentními objekty“. Bez ohledu na složitost taktického plánu umělé inteligence jej lze vždy zahrnout do těchto tří stavů KA, což zajišťuje zjednodušené provádění a udržuje soudržnost akcí NPC v průběhu hry.



Obrázek 1.4: Konečný automat hry F.E.A.R.[17]

Dalším aspektem designu AI, kterou vývojáři implementovali, je kdy využít AI k dosažení požadovaných výsledků a kdy použít důmyslná řešení a úpravy, neboli „padělat“ chytrost AI. Například zatímco jednotliví vojáci ve hře F.E.A.R. nemají povědomí o existenci ostatních, hra vytváří iluzi týmové spolupráce pomocí chytré manipulace. Přidělením samostatných cílů různým postavám, jejichž kombinace simuluje spolupráci, hra naaranžuje scénáře, v nichž například jedna postava poskytuje krycí palbu, zatímco druhá se postaví do boku hráče, aby ho mohli jednodušeji zneškodnit. Ačkoli jsou tyto akce prováděny nezávisle na sobě, jejich synchronizace vytváří dojem koordinovaného manévru. Pro posílení této iluze hra obsahuje dialogy mezi postavami, přestože se navzájem neslyší a neodpovídají si, což přidává další vrstvu realismu a umocňuje hráčův dojem, že bojuje proti koordinovanému týmu.

1.4.3 ME: Shadow of Mordor a ME: Shadow of War

Systém Nemesis[19], představený ve hře Middle-Earth: Shadow of Mordor v roce 2014, který stále patří mezi nejvyspělejší AI ve videohrách. V podstatě funguje jako dynamický generátor padouchů, který vytváří unikátní protivníky s odlišnými vlastnostmi, osobnostmi a rysy. Tito protivníci hráče zaujmou posměšnými průpovídkami a přispívají k příběhu, který se vyvíjí na základě interakcí s hráčem.

V Middle-Earth: Shadow of War je systém Nemesis druhé generace, který rozšiřuje systém z předchozí hry v mnoha ohledech. Nejenže generuje padouchy, ale také spřádá složité příběhy založené na vztazích hráče s těmito orky. Tento systém důmyslně využívá základní hráčské akce, jako je boj, útěk nebo shromažďování informací, jako stavební kameny příběhu a vytváří tak dynamický příběhový zážitek v rámci akční hry v reálném čase.

Klíčem k úspěchu v systému Nemesis je vytvoření zapamatovatelných postav se silnou osobností a jedinečnými vlastnostmi. Díky dialogům, jako jsou například urážky mířené na hráčskou postavu nebo zpívání, ale i díky vizuálním podnětům se každý ork stává osobitým a zapamatovatelným.



Obrázek 1.5: Příklad pevnosti ve hře Shadow of War. Každý ork je unikátní jak vzhledově i chováním. Orkové s modrou lebkou jsou na straně hráče, všichni ostatní jsou nepřátelé. Tento obrázek ukazuje i hierarchii orků, kde ork na nejvyšší pozici je „Overlord“, prostřední pozice je „Warchief“ a orkové před hradbami jsou kapitáni, ale také jsou ve hře orkové, co nemají žádné postavení a jsou to buďto vojáci nebo otroci [18]

Genialita tohoto systému také spočívá v jeho schopnosti prodloužit tyto příběhy za hranice jednotlivých střetnutí a nabídnout alternativní výsledky, jako je ústup, zrada nebo dokonce vzkříšení poražených orků. To zajišťuje, že příběhy přetrvávají a vyvíjejí se na základě akcí a rozhodnutí hráče. Mezi dalšími rozšířeními těchto příběhů jsou vzácné události, jako je zrada „Warchief“ nebo proměna.

Pro podporu těchto vyvíjejících se příběhů hra také obsahuje hierarchii orkské společnosti, která umožňuje povyšování, degradování a posuny moci mezi orky. Navíc se tento systém dynamicky přizpůsobuje na základě výkonu hráče a přináší výzvy nebo příležitosti k udržení příběhového napětí. Systém Nemesis prosperuje na základě zapojení a interpretace hráče, což umožňuje vznik různých příběhů, díky kterým vznikají individuálních zkušenosti a herní prožitky.

Kapitola 2

Strojové učení (Machine Learning - ML)

Strojové učení, jedna z hlavních součástí umělé inteligence, se zabývá vývojem modelů a algoritmů, které umožňují počítačům autonomně se učit z dat a iterativně zvyšovat svůj výkon na základě předchozích zkušeností, a to bez explicitního programování pro každou úlohu.

Zjednodušeně řečeno, strojové učení zahrnuje trénink systémů, které simulují kognitivní procesy podobné lidským, a to prostřednictvím přijímání a analýzy dat. Tato metodika tréninku zahrnuje neustálé zdokonalování prostřednictvím vystavení historickým datům, což vede k opakovanému zlepšování výkonu systému v průběhu času.

Použití strojového učení je zvláště významné v oblastech, kde existuje obrovské množství dat, která vyžadují pomoc při předvídání, což pomáhá při dosahování rychlosti a přesnosti. Tato schopnost usnadňuje práci a ukazuje tak transformační potenciál strojového učení v současných technologických ekosystémech.

Existuje několik typů strojového učení, přičemž každý z nich má zvláštní vlastnosti a aplikace:

1. Supervised Machine Learning („Učení s učitelem“)
2. Unsupervised Machine Learning („Učení bez učitele“)
3. Semi-Supervised Machine Learning („Částečně řízené učení“)
4. Reinforcement Learning („Zpětnovazebné učení“)

2.1 Supervised Machine Learning

„Učení s učitelem“ (SML) [20] představuje algoritmický proces zaměřený na aproximaci základní funkce, která spojuje označená data s jejich odpovídajícími atributy nebo rysy. Tento metodický přístup je založen na využití označených příkladů k rozpoznání složitých vztahů v rámci souborů dat.

Vezměme si typický příklad zahrnující stroj, který má za úkol rozlišovat mezi jablky a hruškami na základě specifikovaných vlastností, jako je např. barva a velikost. Prostřednictvím vystavení mnoha označeným vzorkům ovoce - z nichž každý obsahuje odlišné kombinace barvy a velikosti spolu s odpovídajícími označeními (jablko nebo hruška) - prochází stroj fázi učení, aby zvládl tento klasifikační úkol. Po dokončení procesu učení stroj usiluje o to, aby dokázal určit kategorii (jablko nebo hruška) nového, dosud neviděného ovoce výhradně na základě pozorovaných atributů barvy a velikosti.

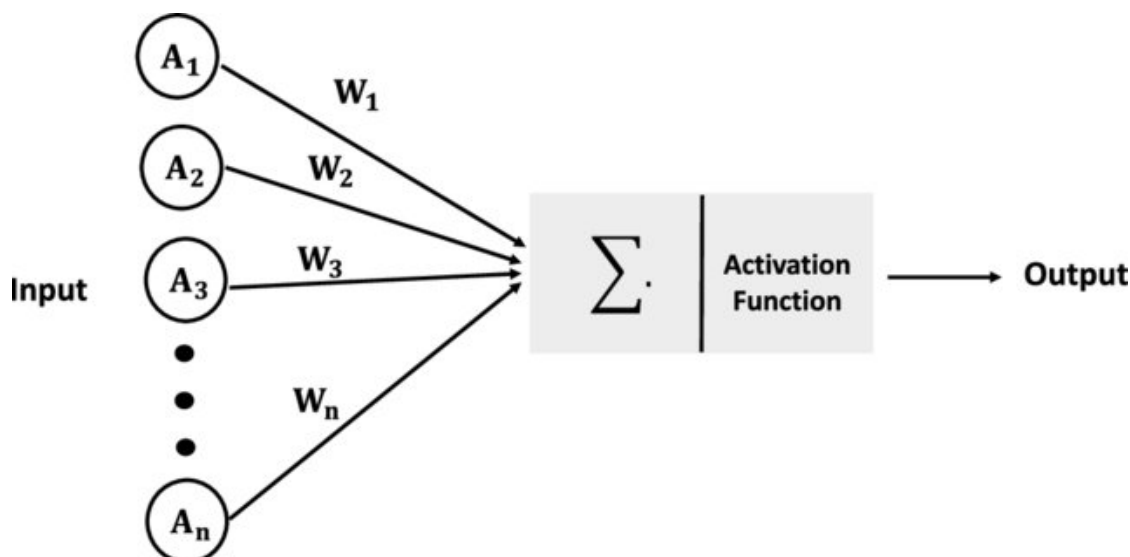
SML je zásadně závislý na přizpůsobeném souboru dat s označenými trénovacími příklady, což je vlastnost, která definuje jeho povahu pod dohledem. Konkrétně toto paradigma učení funguje tak, že dostává tréninkový signál v podobě kontrolovaných značek aplikovaných na datové instance, které jsou charakterizovány odpovídajícími rysy.

Každá datová instance v SML se skládá z dvojice složené z označených výstupů a jim přiřazených vstupních rysů. Primární cíl SML však přesahuje pouhou asimilaci dat - jeho cílem je odvodit funkci, která přesně modeluje základní vztah mezi vstupy a výstupy a která dokáže dobře zobecnit na dosud neznámé dvojice vstupů a výstupů, což je klíčový aspekt označovaný jako zobecnění.

Jako příklad praktického využití SML ve hrách si uveďme tyto scénáře s mapováním vstupů a výstupů:

1. {zdraví hráče, vlastní zdraví, vzdálenost k hráči} $\rightarrow$ {akce (střelba, útěk, nečinnost)}
2. {předchozí pozice hráče, aktuální pozice hráče} $\rightarrow$ {příští pozice hráče}
3. {počet zabití a headshotů, spotřebovaná munice} $\rightarrow$ {hodnocení dovednosti}
4. {skóre, prozkoumaná/neprozkoumaná část mapy} $\rightarrow$ {lokace základny hráče}

Tato mapování ilustrují složité vztahy mezi vstupy a výstupy a ukazují všestrannost a význam SML v různých oblastech, včetně her i mimo ně. Dříve než se budeme věnovat konkrétním algoritmům je nezbytné zdůraznit význam „štítkových“ dat pro utváření výstupních charakteristik a tím pádem vhodný výběr algoritmu SML.



Obrázek 2.1: Ilustrace umělého neuronu. Do neuronu se vkládá n počet vstupních vektorů $\mathbf{A}$ s odpovídajícími váhovými hodnotami $\mathbf{w}$. Neuron zpracovává vstup tak, že vypočítá vážený součet vstupů s odpovídajícími vahami a přidá váhu zkreslení (b): $x \cdot w + b$. Výsledný vzorec se vloží do aktivační funkce (g), jejíž hodnota určuje výstup neuronu.[21]

Algoritmy SML lze rozdělit do dvou hlavních typů podle povahy značkovacích dat (výstupů):

1. **Klasifikace**
2. **Regrese**

2.1.1 Klasifikace

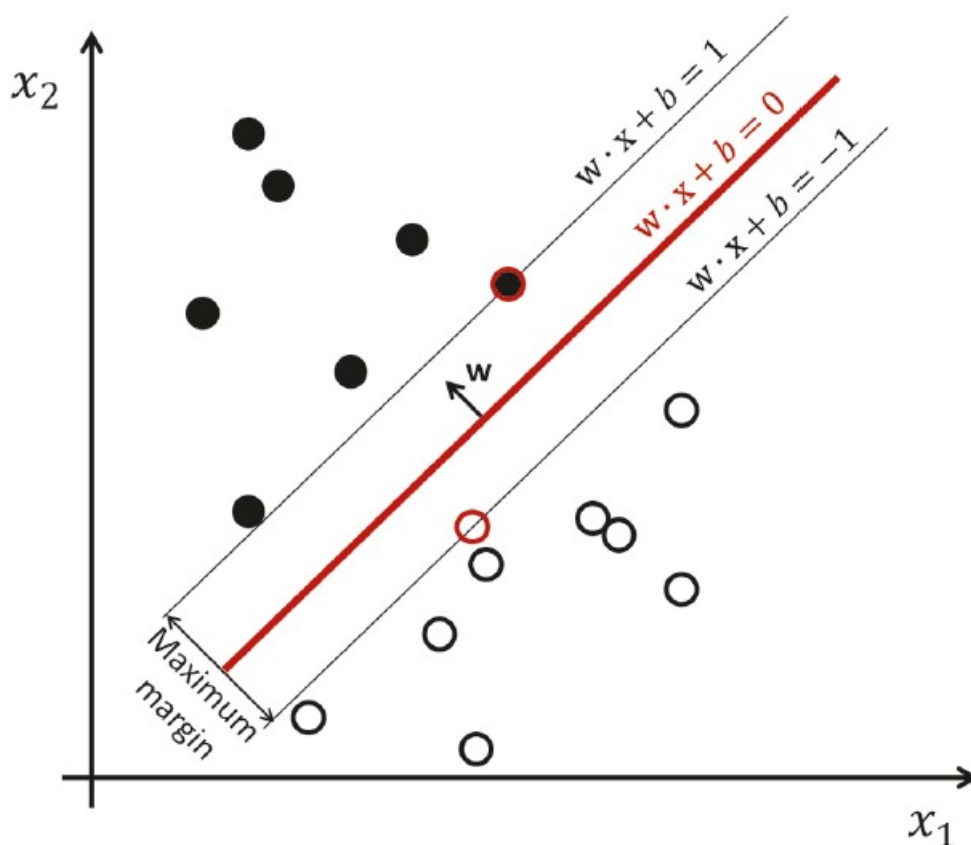
Klasifikace zahrnuje predikci kategoriálních cílových proměnných, které označují diskrétní třídy nebo štítky. Příkladem může být rozlišení e-mailů na poštu a spam, výše zmíněný příklad s jablky a hruškami a posouzení, zda pacient vykazuje vysoké riziko srdečního onemocnění. Úkolem klasifikačních algoritmů je naučit se mapování mezi vstupními znaky a předem definovanými třídami.

Support Vector Machine(Metoda podpůrných vektorů)

Support Vector Machines(SVM)[22] jsou poměrně oblíbené v oblasti algoritmů SML, které se zabývají jak klasifikačními, tak regresními úlohami. SVM fungují jako binární lineární klasifikátory, které jsou vycvičeny tak, aby optimalizovaly rozpětí mezi různými příklady tříd v rámci souboru dat, jako jsou například jablka a hrušky. Stejně jako ostatní metody SML využívají SVM atributy z nových a dosud nezaznamenaných příkladů k předpovědi příslušných tříd.

SVM jsou široce používány v různých oblastech, včetně kategorizace textu, rozpoznávání řeči, klasifikace obrázků a rozpoznávání ručně psaných znaků, a prokazují tak všestrannost a účinnost v různých aplikacích.

Podobně jako umělé neuronové sítě (ANN) vytvářejí SVM ve vstupním prostoru hyperrovinu, která slouží jako funkce f odpovědná za mapování vstupů na cílové výstupy. Na rozdíl od ANN, které minimalizují nesoulad mezi výstupem modelu a cílovým výstupem pomocí gradientního sestupu (zpětného šíření), se však SVM snaží vytvořit hyperrovinu, která maximalizuje rozpětí k nejbližšímu datovému bodu libovolné třídy. Tato hyperplocha s maximální marží účinně odděluje datové body (x_i) třídy od štítků (y_i) označené jako 1 od bodů označených jako -1 v rámci souboru dat obsahujícího n vzorků. Zjednodušeně řečeno, hyperplocha maximalizuje vzdálenost k nejbližšímu datovému bodu (x_i) z kterékoli třídy. Obecně je definována jako $\mathbf{w} \cdot \mathbf{x} - b = 0$, kde $\mathbf{w}$ představuje váhový (normálový) vektor hyperplochy a $\frac{b}{\|\mathbf{w}\|}$ určuje posun nebo vychýlení hyperplochy od počátku, SVM používají vstupní atributy $\mathbf{x}$ z trénovacího souboru dat k vytvoření této hyperplochy (viz obrázek 2.2).



Obrázek 2.2: Příklad hyperplochy s maximální hranicí (červená tlustá čára) a hranicemi (černé čáry) pro SVM, která je natrénována na vzorcích dat ze dvou tříd. Plné a prázdné kruhy odpovídají datům se značkami 1, resp. -1. Klasifikace je v tomto příkladu mapována na dvourozměrný vstupní vektor (x_1, x_2) . Dva vzorky dat na okraji - kruhy znázorněné červeným obrysem - jsou podpůrné vektory.[23]

Formálně vzato je tedy SVM funkce $f(x; w, b)$, která předpovídá cílové výstupy (y) a snaží se:

$$\text{minimalizovat } ||\mathbf{w}'||, \text{ podléhá } y_i(\mathbf{w}' \cdot x_i - b) \geq 1, \text{ pro } i = 1, \dots, n \quad (2.1)$$

Parametry klasifikátoru SVM, váhy (w) a zkreslení (b), určují jeho funkčnost. Datové body (vektory x_i), které se nacházejí nejbližší k určené hyperrovině, se označují jako podpůrné vektory. Daná úloha je považována za řešitelnou, pokud trénovací data vykazují lineární separabilitu, což se často označuje jako klasifikační úloha s tvrdou hranicí (viz obr. 2.2). Pokud data nejsou lineárně separovatelná ("soft-margin"), SVM se místo toho pokusí:

$$\text{minimalizovat } \left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}' \cdot x_i - b)) \right] + \lambda ||\mathbf{w}'||^2 \quad (2.2)$$

Výraz $\lambda ||\mathbf{w}'||^2$ je výsledkem, pokud jsou splněna tvrdá omezení uvedená v rovnici 2.1, což znamená, že všechny datové body jsou správně klasifikovány na pravé straně okraje. Hodnota vypočtená v rovnici 2.2 je přímo úměrná vzdálenosti od okraje pro nesprávně klasifikovaná data, přičemž λ slouží ke kvalitativnímu určení rozsahu, v jakém by se měla velikost okraje zvětšit, a zároveň zajišťuje, že datové body (x_i) zůstanou na správné straně okraje. Volba malé hodnoty λ se blíží klasifikátoru s tvrdou marží pro lineárně separovatelná data.

V konvenční metodice pro trénování klasifikátorů "soft-margin" je proces učení chápán jako problém kvadratického programování, který vyžaduje prozkoumání prostoru parametrů (w a b) s cílem zjistit nejširší marži, která zahrnuje všechny datové body. Mezi alternativní metodiky patří subgradientní sestup a souřadnicový sestup.

Kromě lineární klasifikace SVM umožňuje i nelineární klasifikaci s využitím různých nelineárních jader. Tato jádra usnadňují mapování vstupního prostoru na prostor vyšších rozměrů, čímž rozšiřují použitelnost SVM. Navzdory této transformaci zůstává základní cíl SVM zachován, i když s nahrazením bodových součinů nelineárními funkcemi jádra. Tato úprava umožňuje algoritmu vymezit hyperplochu s maximální hranicí v transformovaném prostoru příznaků. Mezi nejznámější jádra používaná ve spojení s SVM patří polynomiální funkce, Gaussovy radiální báze funkce a hyperbolické tangenciální funkce.

Zatímco původně byly SVM navrženy pro řešení binární klasifikace, vznikla řada iterací SVM pro klasifikaci a regresi více tříd. SVM mají oproti alternativním SML několik výhod. Zejména vykazují účinnost při řešení problémů spojených s velkými, ale řídkými soubory dat, protože jejich účinnost závisí pouze na podpůrných vektorech pro konstrukci hyperploch. Kromě toho si SVM dovedou poradit s rozsáhlými prostory příznaků, protože složitost jejich učební úlohy není ovlivněna dimenzionalitou prostoru příznaků. Dále SVM představují jednoduchý konvexní optimalizační problém, který zajišťuje konvergenci k singulárnímu globálnímu řešení. A konečně, klasifikační přístup se "soft-margin" umožňuje snadnou kontrolu nad tendencemi k nadměrnému přizpůsobování.

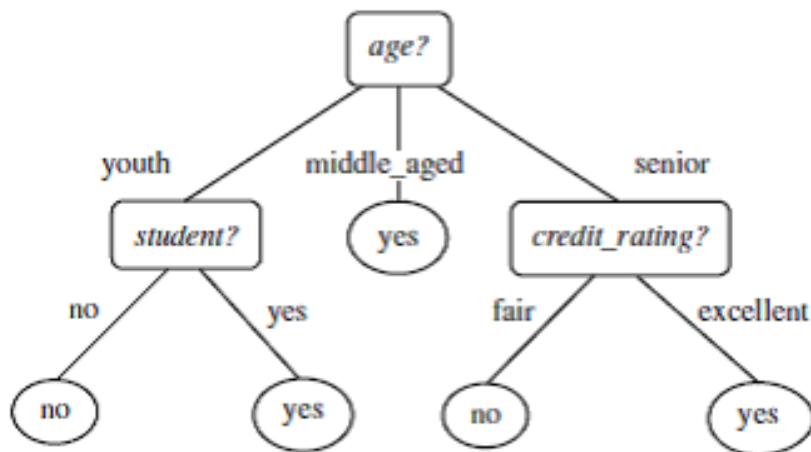
2.1.2 Regrese

Na rozdíl od klasifikace se regrese zaměřuje na předpovídání spojitých cílových proměnných, které představují číselné hodnoty. Jedná se například o předpovídání ceny domu s ohledem na faktory jako je velikost, poloha a vybavení, nebo o odhad prodeje výrobků. Regresní algoritmy jsou navrženy tak, aby se naučily mapování mezi vstupními prvky a spojitými číselnými hodnotami.

Decision Trees(Rozhodovací stromy)

V rámci rozhodovacích stromů(DT)[24] má zkoumaná funkce f strukturu rozhodovacího stromu, který mapuje atributy pozorování dat na odpovídající cílové hodnoty. Vstupy jsou reprezentovány jako uzly, zatímco výstupy jsou ve stromu znázorněny jako listy. Potenciální hodnoty každého vstupního uzlu jsou vymezeny různými větvemi vycházejícími z tohoto uzlu. Analogicky k jiným SML jsou rozhodovací stromy rozděleny do kategorií podle typu výstupních dat, která se snaží naučit. Rozhodovací stromy lze zejména členit na klasifikační nebo regresní podle toho, zda cílový výstup obsahuje konečnou množinu hodnot, nebo spojitou množinu spojitých (intervalových) hodnot.

Příklad rozhodovacího stromu je znázorněn na obrázku 2.3. V tomto stromovém zobrazení odpovídají uzly vstupním atributům a větve se rozšiřují na potomky pro každou myslitelnou hodnotu každého vstupního atributu. Následně listy označují výstupní hodnoty, v tomto případě ohraničují výsledky zda je jedinec studentem nebo zda má příznivé kreditní skóre. Tyto výsledky jsou závislé na hodnotách vstupních atributů, jak je určuje cesta, kterou procházíme od kořene k listu.



Obrázek 2.3: Příklad DT: Vzhledem k věku, statusu studenta a skóre kreditní karty strom předpovídá, zda si osoba koupí počítač. Uzly stromu (zaoblené obdélníky) představují atributy dat neboli vstupy, zatímco listy (ovály) představují cílové hodnoty neboli výstupy. Větve stromu představují možné hodnoty příslušného nadřazeného uzlu stromu.[25]

Cílem DT je formulovat mapování reprezentované jako stromový model, který je schopen předpovídat cílové výstupní hodnoty na základě více vstupních atributů. Základní a široce používaná metoda učení DT z dat je charakterizována hladovým algoritmem. Tento algoritmus předpokládá, že vstupní atributy i cílové výstupy mají konečné diskrétní oblasti a vykazují kategoriální charakteristiky. V případech, kdy vstupy nebo výstupy obsahují spojitě hodnoty, může konstrukci stromu předcházet diskretizace. Iterační konstrukce stromu zahrnuje rozdělení dostupné trénovací množiny dat na podmnožiny, která se řídí výběrem atributů. Tento rekursivní proces se odvíjí atribut po atributu a postupně zpřesňuje strukturu stromu.

Různé iterace výše uvedeného postupu vedou k různým algoritmům rozhodovacích stromů. V oblasti učení rozhodovacích stromů se zejména používají dvě významné varianty: Iterative Dichotomiser 3 (ID3) a jeho nástupce C4.5.

2.2 Unsupervised Machine Learning

„Učení bez učitele“ (UML) je technika strojového učení, při které algoritmus samostatně odhaluje vzory a vztahy v datech, jež nejsou opatřena žádnými štítky nebo etiketami. Na rozdíl od řízeného učení, kde algoritmus pracuje s předem označenými výstupy, se v bezdohorovém učení algoritmus obejde bez těchto předem definovaných cílů. Hlavním cílem UML je často identifikace skrytých vzorců, podobností či klastrů v rámci dat, které mohou sloužit k různým účelům, včetně průzkumu dat, vizualizace, redukce dimenzionality a dalších aplikací.

Tento koncept lze lépe představit prostřednictvím konkrétního příkladu. Představme si, že máme dataset obsahující informace o nákupech, které jsme uskutečnili v obchodě. Např. pomocí techniky shlukování může algoritmus seskupit podobné nákupní chování mezi námi a ostatními zákazníky, což umožňuje odhalit potenciální zákazníky bez nutnosti použití předdefinovaných štítků. Tento typ analýzy poskytuje cenné poznatky pro podniky, neboť umožňuje efektivněji zacílit na specifické zákaznické segmenty a zároveň identifikovat anomálie nebo odlehlé hodnoty v datech.

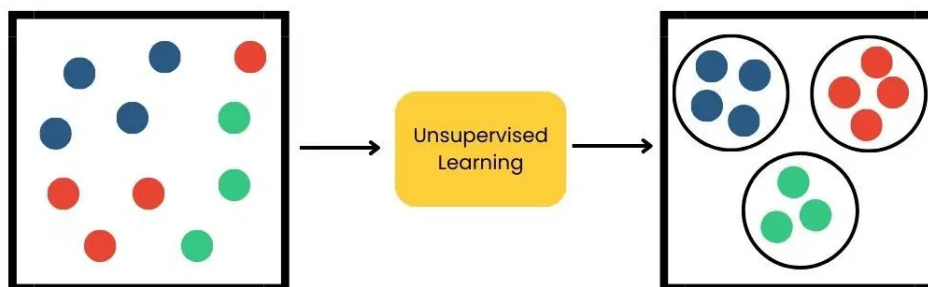
Algoritmy UML lze rozdělit do dvou hlavních typů:

1. **Shlukování**
2. **"Těžba častých vzorů" neboli asociace**

2.2.1 Shlukování

Shlukování[26] představuje úlohu učení bez učitele, která se zaměřuje na identifikaci neznámých skupin v souboru dat. Cílem je zajistit, aby datové body v rámci stejné skupiny nebo shluku vykazovaly vysokou podobnost a zároveň se lišily od dat v jiných shlucích.

Tato technika se široce uplatňuje v různých oblastech, včetně detekce skupin dat ve více atributech, a také v úlohách redukce dat, jako je komprese dat, vyhlazení šumu, detekce odlehlých hodnot a rozdělení datové sady. Shlukování má značný význam zejména v herním průmyslu, kde se využívá pro modelování hráčů, vývoj herních strategií a generování obsahu.



Obrázek 2.4: Jednoduchá interpretace shlukování.[27]

Podobně jako klasifikace zahrnuje shlukování přiřazení dat ke třídám, nicméně značky tříd nejsou předem známy a shlukovací algoritmy se je snaží odhalit iterativním vyhodnocováním jejich kvality. Protože správné shluky nejsou předem definovány, podobnost a nepodobnost závisí pouze na attributech analyzovaných dat. Efektivní shluky jsou definovány dvěma základními vlastnostmi:

1. Vysoká podobnost uvnitř shluku, známá také jako vysoká kompaktnost
2. Nízká podobnost mezi shluky neboli dobrá separace

Běžným měřítkem kompaktnosti je průměrná vzdálenost mezi každým vzorkem ve shluku a jeho nejbližším reprezentativním bodem, například centroidem, jak se využívá v algoritmu k-means. Mezi míry separace patří jednoduchá vazba a úplná vazba:

1. Jednoduchá vazba:
 - Označuje nejmenší vzdálenost mezi kterýmkoli vzorkem v jednom shluku a kterýmkoli vzorkem v jiném shluku.
2. Úplná vazba:
 - Označuje naopak největší vzdálenost mezi kterýmkoli vzorkem v jednom shluku a kterýmkoli vzorkem v jiném shluku.

Ačkoli kompaktnost a separace jsou objektivními měřítky pro posouzení platnosti shluků, je nezbytné si uvědomit, že nemusí nutně ukazovat na smysluplnost shluků.

Kromě už zmíněných metrik platnosti jsou shlukovací algoritmy charakterizovány funkcí příslušnosti a postupem vyhledávání. Funkce příslušnosti určuje strukturu shluků ve vztahu ke vzorkům dat, zatímco postup vyhledávání popisuje strategii použitou ke shlukování dat vzhledem ke konkrétní funkci příslušnosti a mtrice platnosti. Některé strategie například zahrnují rozdělení všech datových bodů do shluků najednou, jako je tomu u k-means, zatímco jiné zahrnují rekurzivní slučování nebo rozdělování shluků, jako je tomu u hierarchického shlukování. Shlukování lze provádět pomocí široké škály algoritmů, včetně už zmíněného hierarchického shlukování a k-means ale také pomocí k-medoidů, DBSCAN a samoorganizujících se map. Tyto algoritmy se liší způsobem, jakým definují shluk, a metodami, které používají k jeho vytvoření. Výběr vhodného shlukovacího algoritmu a s ním spojených parametrů, jako je například použitá funkce vzdálenosti nebo očekávaný počet shluků, závisí na konkrétních cílech studie a povaze dostupných dat. V následujícím oddíle jsou uvedené algoritmy shlukování, které jsou považovány za nejefektivnější pro studium umělé inteligence ve hrách.

K-means shlukování

K-means[28] je vektorová kvantizační technika, která je díky své efektivní rovnováze mezi jednoduchostí a výkonností považována za nejoblíbenější shlukovací algoritmus. Využívá přímočarý přístup k rozdělení dat, kdy je databáze objektů rozdělena do k shluků. Cílem je minimalizovat součet čtvercových euklidovských vzdáleností mezi každým datovým bodem a příslušným středem shluku neboli centroidem - tato vzdálenost se běžně označuje jako kvantizační chyba.

V k-means je každý shluk reprezentován jedním bodem, známým jako centroid, přičemž každý vzorek dat je přiřazen k nejbližšímu centroidu. Samotný centroid se vypočítá jako průměr datových vzorků v rámci shluku. Metrikou platnosti uvnitř shluku, kterou k-means používá, je průměrná vzdálenost každého datového bodu od příslušného centroidu. Zpočátku jsou vzorky dat náhodně přiřazeny ke shlukům a algoritmus pak iterativně střídavě přiřazuje datové body ke shlukům a aktualizuje centroidy na základě nového složení shluků. Základní kroky algoritmu jsou následující:

- Pro dané k :
 1. Náhodně rozděl datové body do k neprázdných shluků.
 2. Vypočti polohy centroid shluků aktuálního rozdělení.
 3. Přiřaď každý datový bod ke shluku s nejbližším centroidem. Pokud se přiřazení nemění, ukončíme výpočet, v opačném případě přejdeme ke kroku 2.

Ačkoli je metoda k-means velmi oblíbená díky své jednoduchosti, má několik významných omezení. Zaprvé je použitelná pouze pro datové objekty ve spojitém prostoru. Za druhé, počet shluků, k , musí být předem určen. a za třetí, metoda k-means má problémy s identifikací shluků s nekonvexními tvary, protože je omezena na hledání hyperkulových shluků.

Kromě toho je algoritmus k-means citlivý na odlehlé hodnoty, neboli datové body s mimořádně velkými nebo malými hodnotami mohou zkreslit celkové rozložení dat, a tím ovlivnit výkonnost algoritmu.

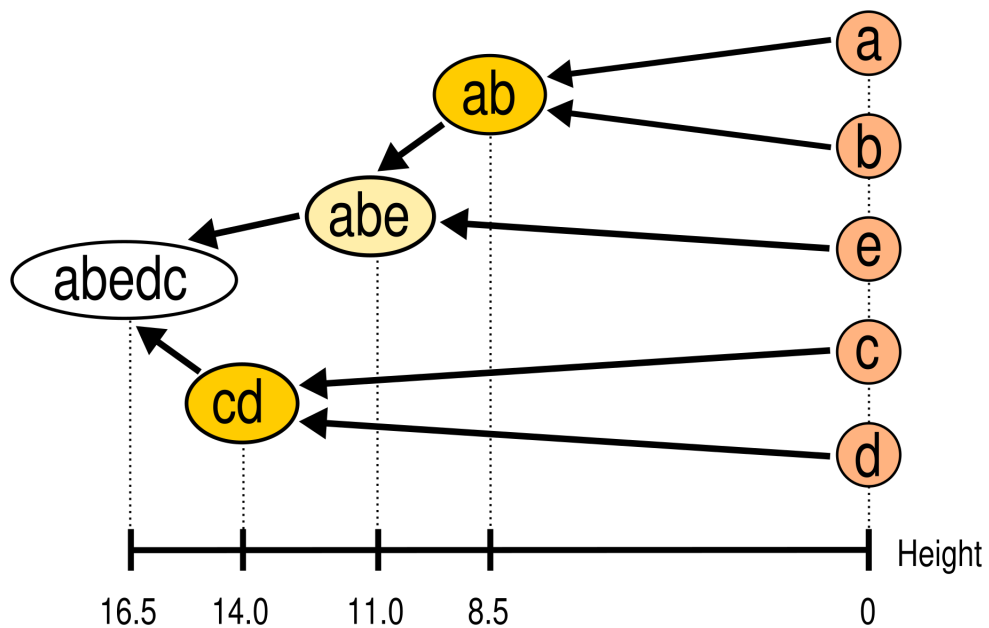
Hierarchické shlukování

Metody shlukování, jejichž cílem je vytvořit hierarchii shluků, se řadí do kategorie hierarchického shlukování[29]. Obecně existují dvě základní strategie: aglomerativní a rozdělovací. Aglomerativní strategie vytváří hierarchie způsobem zdola nahoru postupným slučováním datových bodů, zatímco divizivní strategie vytváří hierarchie postupným rozdělováním souboru dat způsobem shora dolů. Obě strategie jsou ze své podstaty chamtivé. Hierarchické shlukování se spoléhá na matici vzdáleností, která řídí proces shlukování, ať už aglomerativního, nebo divizivního. Na rozdíl od metody k-means tato metoda nevyžaduje, aby byl předem určen počet shluků k , ale vyžaduje podmínku ukončení.

Pro ilustraci uvedeme základní kroky algoritmu aglomerativního shlukování, které jsou následující:

- Pro dané k :
 1. Vytvořte jeden shluk pro každý vzorek dat.
 2. Najděte nejkratší euklidovskou vzdálenost mezi dvěma body, které nejsou ve stejném shluku.
 3. Slučte shluky obsahující tyto dva vzorky. Pokud existuje k shluků, ukončíme výpočet, v opačném případě přejdeme ke kroku 2.

Při rozdělovacím hierarchickém shlukování patří všechny datové body zpočátku do jednoho shluku, který je pak iterativně rozdělen, dokud se každý datový bod nenachází ve vlastním shluku. Tento proces se řídí strategií rozdělení, jako je například Divisive Analysis Clustering (DIANA), nebo využívá jiný shlukovací algoritmus k rozdělení dat do dvou shluků, například 2-means. Jak jsou shluky rekurzivně slučovány nebo rozdělovány, lze výsledné shluky vizualizovat prostřednictvím několika úrovní vnořeného rozdělení. Výsledkem je v podstatě stromová reprezentace shluků, známá jako dendrogram. Konečné shlukování se určí rozříznutím dendrogramu na požadované úrovni čtvercové euklidovské vzdálenosti.

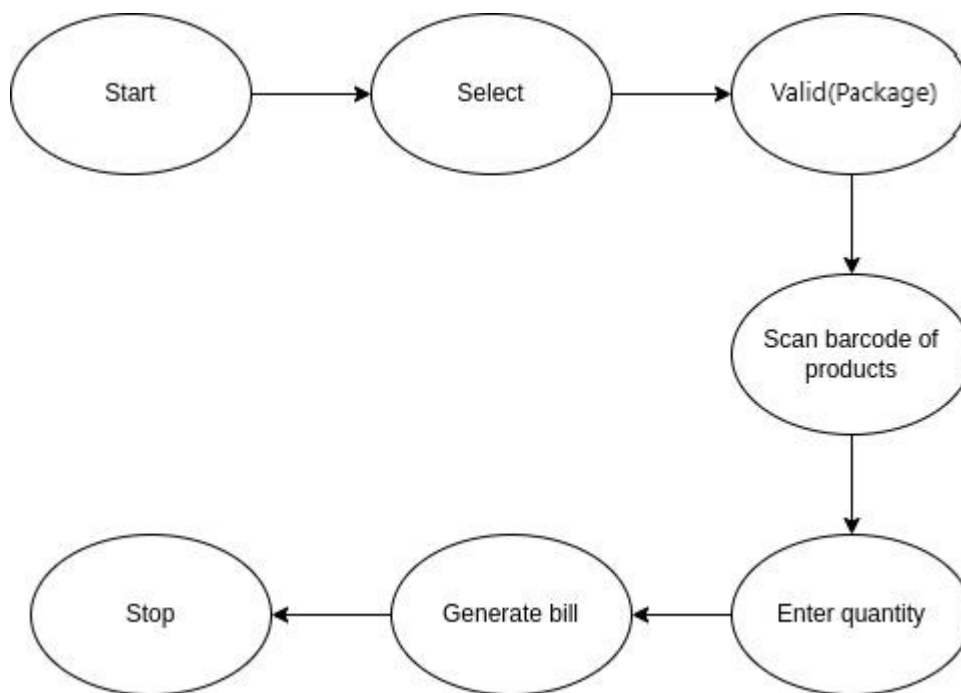


Obrázek 2.5: Dendrogram hierarchického shlukování (UPGMA) s výškou uzlů.[30]

Hierarchické shlukování představuje shluky jako soubor datových vzorků v jejich rámci, což znamená, že datový vzorek je seskupen se svým nejbližším sousedem. Naproti tomu k-means definuje každý shluk pomocí centroidu, takže vzorek dat patří do shluku spojeného s nejbližším centroidem. Kromě toho se při měření platnosti shluků aglomerativní shlukování spoléhá na nejkratší vzdálenost mezi kterýmkoli vzorkem v jednom shluku a kterýmkoli vzorkem v jiném shluku, zatímco k-means používá průměrnou vzdálenost každého vzorku od centroidu. Díky těmto rozdílným algoritmickým charakteristikám může hierarchické shlukování pojmout data s různými spojitými tvary, zatímco k-means je omezeno na hyperkulaté shluky.

2.2.2 „Těžba častých vzorů“ neboli asociace(FPM)

FPM[31] zahrnuje řadu technik určených k odhalování častých vzorů a struktur v datech. Tyto vzory mohou zahrnovat sekvence a množiny položek. Původně vyvinuté pro dolování asociačních pravidel, FPM se snaží identifikovat atributy dat, které se často vyskytují společně, a vytvářet tak mezi nimi podmíněná pravidla. V kontextu herní umělé inteligence jsou důležité zejména dva specifické typy dolování častých vzorů: dolování častých množin položek a dolování častých sekvencí. Frequent item-set mining se zaměřuje na odhalování struktur mezi datovými atributy bez ohledu na vnitřní pořadí, zatímco frequent sequence mining se snaží odhalit vzory založené na vnitřním časovém pořadí datových atributů. Ačkoli FPM spadá pod neřízené učení, liší se jak svými cíli, tak použitými algoritmy.



Obrázek 2.6: Jednoduchá vizualizace FPM.

Mezi široce používané a škálovatelné metody pro dolování častých vzorů patří algoritmus Apriori pro dolování množin položek a GSP pro dolování sekvencí. V následujících dvou částí tyto algoritmy popíšeme jako reprezentativní algoritmy pro dolování častých množin položek a dolování častých sekvencí.

Apriori

Apriori[32] je algoritmus určený pro dolování častých položek. Je vhodný pro analýzu datových souborů, které obsahují množiny instancí, známé také jako transakce, z nichž každá obsahuje kolekci položek nebo množiny položek. Transakce mohou například představovat knihy zakoupené zákazníkem Amazonu nebo aplikace pořízené uživatelem chytrého telefonu. Algoritmus je poměrně jednoduchý a funguje následovně: při zadání předem definované prahové hodnoty nazývané podpora (T) identifikuje Apriori množiny položek, které jsou podmnožinami alespoň T transakcí v databázi. Apriori se v podstatě snaží odhalit všechny množiny položek, které splňují nebo překračují minimální prahovou hodnotu podpory, která představuje minimální četnost, s níž se daná množina položek objevuje v souboru dat.

Pro ilustraci použití Apriori v herním kontextu uvádíme níže reprezentativní seznam událostí čtyř hráčů v online hře na hraní rolí:

- <Dosažena 11. úroveň; Odemčeno více než 50 % úspěchů; Zakoupen démonický luk>
- <Dosažena 11. úroveň Zakoupen démonický luk>
- <Odemčeno více než 50 % úspěchů Zakoupen démonický luk Nalezen pramen života>
- <Dosažena 11. úroveň Nalezen pramen života Odemčeno více než 50 % úspěchů Zakoupen démonický luk>

Vzhledem k příkladovému souboru dat a za předpokladu prahové hodnoty podpory 3 lze identifikovat následující soubory obsahující pouze jednu položku: <Dosažena 11. úroveň>, <Odemčeno více než 50 % úspěchů> a <Zakoupen démonický luk>. Pokud místo toho hledáme sady 2-položek se stejným prahem podpory, najdeme např. <Dosažena 11. úroveň, Zakoupen démonický luk>, protože tři transakce obsahují obě tyto položky. Žádné delší sady předmětů nejsou k dispozici (tj. nejsou časté) pro počet podpor 3. Tento postup lze použít pro libovolný práh podpory k odhalení častých sad předmětů.

Zobecněné sekvenční vzory(GSP)

Algoritmy pro dolování častých položek jsou nedostatečné, pokud je pro analýzu rozhodující posloupnost událostí. Pokud soubor dat obsahuje události uspořádané do uspořádaných sekvencí, jako jsou časové sekvence nebo časové řady, je nutný přístup založený na dolování častých sekvencí. Problém dolování častých sekvencí lze stručně definovat jako identifikaci často se vyskytujících následných sekvencí v rámci dané sekvence nebo množiny sekvencí.

Formálněji řečeno, v souboru dat, kde každý vzorek představuje posloupnost událostí, známou jako sekvence dat, je sekvenční vzor definován jako posloupnost událostí, která se kvalifikuje jako častá sekvence, pokud se ve vzorcích objevuje pravidelně. Častá sekvence je charakterizována jako sekvenční vzor, který je podporován alespoň minimálním počtem datových sekvencí, jak je určeno prahem nazývaným minimální hodnota podpory. Datová sekvence podporuje sekvenční vzor, pokud obsahuje všechny události vzoru ve stejném pořadí. Například datová posloupnost < v, w, x, y, z > podporuje vzor < v, z >. Podobně jako při vytěžování častých množin položek se počet datových posloupností, které podporují sekvenční vzor, označuje jako počet podpor.

Algoritmus GSP (Generalized Sequential Patterns)[33] je široce používaná technika pro dolování častých sekvencí z dat. Algoritmus GSP začíná identifikací častých sekvencí sestávajících z jediné události, známých jako 1-sekvence. Tyto sekvence se pak samy spojí a vytvoří všechny možné kandidáty na 2-sekvence, pro které se vypočítá počet podpor. Sekvence, které splňují práh četnosti, se následně samy spojí a vytvoří kandidáty na 3-sekvence. Tento proces pokračuje, přičemž algoritmus v každém kroku postupně zvyšuje délku sekvence, dokud nejsou vygenerováni žádní další kandidáti. Základním principem algoritmu je, že pokud je sekvenční vzor častý, pak musí být časté i všechny jeho sousední podvzory.

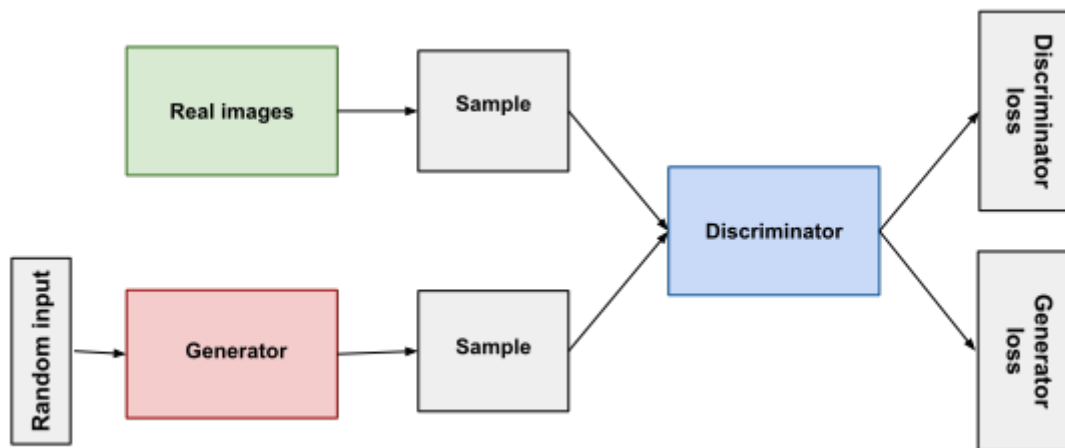
2.3 Semi-Supervised Machine Learning

„Částečně řízené učení“ je typ strojového učení, který překlenuje mezeru mezi řízeným a neřízeným učení tím, že využívá jak označená, tak neoznačená data. Tato metoda je výhodná zejména v případech, kdy je získávání označených dat nákladné a vyžaduje specializované dovednosti nebo značné úsilí. Integrací omezeného množství označených dat s větším množstvím neoznačených dat zvyšuje polopřímé učení výkonnost a efektivitu modelu ve scénářích, kde je úplné označování nepraktické.

Tyto techniky se používají při práci se soubory dat, které obsahují malé množství označených dat a velkou část neoznačených dat. V takových případech lze k předpovědi štítků pro neoznačená data použít neřízené metody, které se pak použijí jako vstupy pro řízené techniky. Tento přístup je zvláště užitečný pro soubory obrazových dat, kde je běžné, že ne všechny obrazy jsou označeny. Nejznámějším typem S-SML je GAN síť.

2.3.1 Generativní adverzní síť (GAN)

GAN[34] je strojového učení určené ke generování nových vzorků dat, které jsou podobné danému souboru dat. Síť GAN, které představil Ian Goodfellow se svými kolegy v roce 2014, se skládá ze dvou neuronových sítí, známých jako generátor a diskriminátor, které jsou trénovány současně prostřednictvím procesu adverzního učení.



Obrázek 2.7: Jednoduchá vizualizace GAN sítě. [35]

Hlavním úkolem generátorové sítě je vytvářet syntetické vzorky dat, které napodobují rozložení skutečných dat. Začíná s náhodným šumem a transformuje tento šum na data, která se podobají trénovací množině. Naopak diskriminační síť funguje jako klasifikátor, který rozlišuje mezi vzorky skutečných dat a vzorky vytvořenými generátorem. Během tréninku generátor zlepšuje svou schopnost vytvářet reálná data tím, že dostává zpětnou vazbu od diskriminátoru, zatímco diskriminátor zlepšuje svou schopnost rozlišovat mezi skutečnými a syntetickými daty.

Trénování pokračuje, dokud generátor nevytvoří vzorky, které jsou podle diskriminátoru nerozlišitelné od skutečných dat. Tato interakce je formalizována jako minimaxová hra, kde generátor usiluje o minimalizaci pravděpodobnosti, že diskriminátor správně identifikuje generované vzorky, zatímco diskriminátor usiluje o maximalizaci své klasifikační přesnosti. GAN jsou známé zejména pro svou schopnost generovat syntézy obrazu nebo text na obraz.

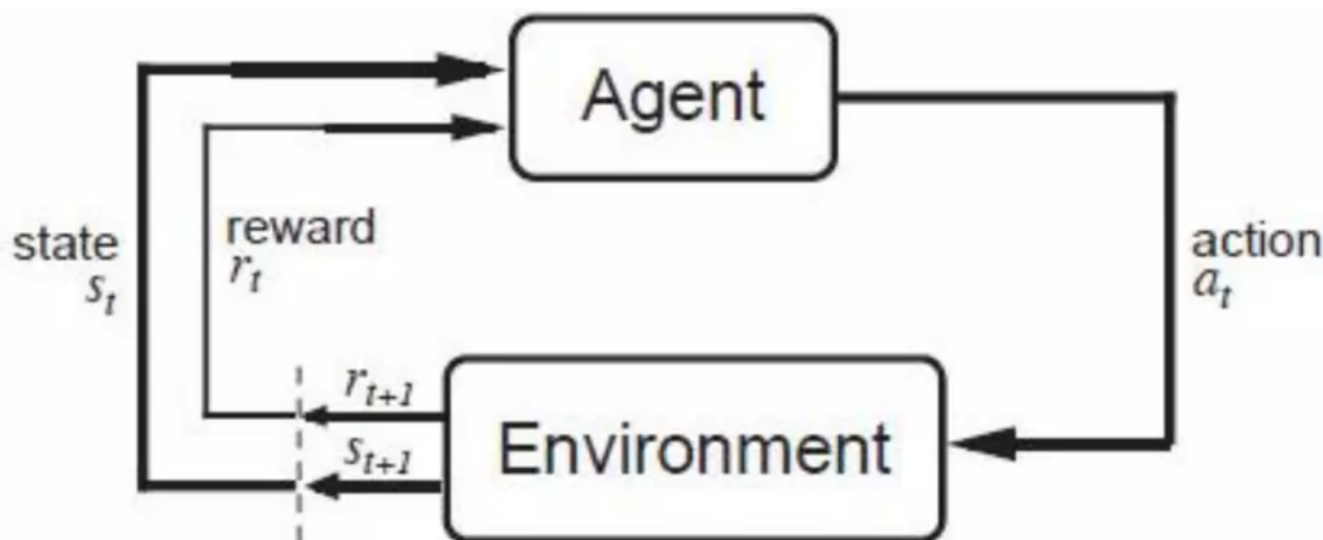
Navzdory své účinnosti se GAN potýkají s problémy, jako je zhroucení režimu, kdy generátor vytváří omezené množství vzorků, a nestabilita trénování.

2.4 Reinforcement Learning

„Zpětnovazebné učení“ (RL) [36] je paradigma strojového učení inspirované behavioristickou psychologií, zejména procesem, při kterém se lidé a zvířata učí činit rozhodnutí na základě odměn (pozitivních nebo negativních) ze svého okolí. Na rozdíl od učení pod dohledem, kdy jsou poskytovány příklady správného chování, funguje posilovací učení prostřednictvím interakce mezi agentem a jeho prostředím, podobně jako evoluční přístupy učení. V každém čase t se agent nachází v určitém stavu s a musí zvolit akci a z množiny možných akcí, které jsou v tomto stavu k dispozici. V reakci na to prostředí poskytuje okamžitou odměnu r . V průběhu času se agent díky neustálé interakci s prostředím naučí vybírat akce, které maximalizují kumulativní odměnu, kterou obdrží. RL bylo zkoumáno z různých akademických hledisek, včetně operačního výzkumu, teorie her, teorie informace a genetických algoritmů. Efektivně se využívá ve scénářích, které vyžadují rovnováhu mezi dlouhodobými a krátkodobými odměnami, například v aplikacích pro řízení robotů.

Cílem agenta při RL učení je určit takovou politiku (π), která maximalizuje dlouhodobou odměnu, například očekávanou kumulativní odměnu. Politika je strategie, kterou agent používá k výběru akcí na základě svého aktuálního stavu. Pokud je známa nebo naučena hodnotová funkce akcí, je optimální politika (π^*) určena výběrem akce s nejvyšší hodnotou. Interakce mezi agentem a prostředím probíhají v diskrétních časových krocích a jsou modelovány jako Markovův rozhodovací proces (MDP). MDP je definován takto:

- **S**: Soubor stavů $\{s_1, \dots, s_n\}$, které představují různé podmínky prostředí na základě informací agenta.
- **A**: Množina akcí $\{a_1, \dots, a_m\}$, představující možné akce, které může agent provést v každém stavu.
- **P**(**s**, s^* , **a**): Pravděpodobnost přechodu ze stavu s do stavu (s^*) při dané akci a podle Markovovy vlastnosti, která znamená, že budoucí stavy závisí pouze na aktuálním stavu.
- **R**(**s**, s^* , **a**): Funkce odměny, která poskytuje okamžitou odměnu r získanou při přechodu ze stavu s do stavu (s^*) po provedení akce a .



Obrázek 2.8: Jednoduchá vizualizace RL myšlení. [37]

Model světa při RL je definován pomocí $\mathbf{P}$ a $\mathbf{R}$, které představují dynamiku prostředí a dlouhodobé odměny spojené s každou politikou. Pokud je model světa znám, není třeba odhadovat pravděpodobnosti přechodu nebo funkce odměn, což umožňuje vypočítat optimální politiku přímo pomocí technik založených na modelu, jako je dynamické programování. Pokud však model světa není znám, je třeba přechodové funkce a funkce odměn aproximovat pomocí naučených odhadů budoucích odměn za provedení akce a ve stavu s . Politika se pak odvozuje z těchto odhadů. Tento proces učení usnadňují bezmodelové metody, jako je prohledávání Monte Carlo a učení pomocí časových rozdílů.

Klíčovou výzvou v RL je dosažení správné rovnováhy mezi **využíváním** aktuálně získaných znalostí a **prozkoumáváním** neprobádaných oblastí v rámci prohledávaného prostoru. Strategie, které se spoléhají pouze na náhodný výběr akce nebo vždy vybírají akci s nejvyšší vnímanou odměnou mají tendenci dosahovat ve stochastických prostředích špatných výsledků. Pro řešení této rovnováhy byly navrženy různé metody, přičemž jednou z nejúčinnějších a nejpoužívanějších je strategie ϵ -greedy. Tento přístup, řízený parametrem $\epsilon \in [0,1]$, nařizuje, že agent RL vybírá akci, u níž očekává, že přinese nejvyšší budoucí odměnu s pravděpodobností $1-\epsilon$, zatímco s pravděpodobností ϵ vybírá akci náhodně.

Problémy RL lze rozdělit na **epizodické** a **inkrementální** typy. V epizodickém RL je algoritmus trénován offline v rámci konečného horizontu, který zahrnuje více trénovacích instancí. Posloupnost stavů, akcí a signálů odměn shromážděných v tomto konečném období se označuje jako epizoda. Příkladem epizodického RL jsou metody Monte Carlo, které se spoléhají na opakované náhodné vzorkování. Naopak v inkrementálním RL učení probíhá online, aniž by bylo omezeno konkrétním horizontem. Významným příkladem algoritmů v rámci kategorie inkrementálního RL je učení pomocí časového rozdílu (Temporal Difference, TD).

Dalším důležitým rozdílem v posilování učení (RL) je rozdíl mezi algoritmy **off-**

policy a **on-policy**. Off-policy aproximuje optimální politiku nezávisle na konkrétních akcích agenta. Například, jak si řekneme v další kapitole, Q-learning je off-policy algoritmus, protože odhaduje očekávaný výnos pro dvojice stav-akce za předpokladu, že je dodržována chamtivá politika. Naproti tomu algoritmus RL on-policy aproximuje politiku ve spojení s akcemi agenta, a to i během fází průzkumu.

A nyní poslední dva pojmy. **Bootstrapping** je klíčový koncept v RL, který rozlišuje algoritmy podle toho, jak optimalizují hodnoty stavu. Zahrnuje aktualizaci odhadu hodnoty stavu pomocí odhadů následujících stavů, což umožňuje průběžné zpřesňování. Jak učení pomocí časového rozdílu (Temporal Difference, TD), tak dynamické programování využívají bootstrapping k aktualizaci hodnot stavu na základě zkušeností. Oproti tomu metody Monte Carlo nepoužívají bootstrapping a místo toho se učí každou hodnotu stavu nezávisle. V RL je klíčový koncept **zálohování**, který zahrnuje práci zpětně od budoucího stavu k aktuálnímu stavu a zahrnuje mezistavy. Operace zálohování se liší hloubkou, od jednoho kroku až po úplné zálohování, a rozsahem, od několika vybraných vzorových stavů až po komplexní analýzu.

Na základě výše uvedených kritérií lze identifikovat tři hlavní typy algoritmů posilování učení:

1. Dynamické programování:

- Vyžaduje znalost modelu světa (P a R) a počítá optimální politiku pomocí bootstrappingu.

2. Monte Carlo metody:

- Nevyžadují znalost modelu světa. Algoritmy této kategorie, jako je například Monte Carlo Tree Search (MCTS), jsou vhodné pro offline (epizodické) trénování a učí se pomocí zálohování v rozsahu vzorku a plné hloubky. Na rozdíl od jiných metod techniky Monte Carlo nepoužívají bootstrapping.

3. Temporal Difference učení:

- Podobně jako u metod Monte Carlo není vyžadována znalost modelu světa a je třeba jej odhadnout. Algoritmy tohoto typu, jako je Q-learning, se učí ze zkušeností pomocí bootstrappingu a různých záložních technik.

Kapitola 3

Návrh autonomního agenta

3.1 Prostředí pro agenta

K vizualizaci autonomního agenta lze přistupovat různými způsoby, od použití vysokoúrovňových herních engineů nebo lze od základu si postavit vlastní engine, pomocí kterého poté lze vytvořit vizualizér. Jako praktický a realistický příklad využijeme Unreal Engine, který se velice často používá k vytvoření 3D akčních her. V této vizualizaci navrhne prostředí které bude obsahovat několik překážek či budov, které budou představovat dodatečnou obtížnost pro nepřátelského, ale i přátelského agenta při učení.

V Unreal Engine vytvoříme mapu, která bude obsahovat část rozpadlého města, kde se budou nacházet různé budovy, ale i park a různé ostatní překážky pro hráče, přátelské či nepřátelské agenty. Mapa bude naplněna převážně prostředky získanými z tržiště Unreal Marketplace, ale také prostředky vlastní výroby. Mapa bude obsahovat počáteční oblast, kde hráč bude začínat, ale kde zároveň se budou nacházet prostředky, pomocí kterých se bude snažit přežít s pomocí přátelských agentů proti agentům nepřátelských.

Herní mapa bude obsahovat různorodé oblasti, jako např. otevřenější část, ale také uzavřenější část obklopenou budovami. Uzavřená oblast nabídne strategické výzvy a výhody, například krytí poskytované různými objekty, které lze takticky využít během střetů. Některé části mapy budou také obsahovat nebezpečné prvky, které mohou způsobit poškození, pokud se hráč nebo agenti přiblíží k tomuto nebezpečnému prvku.

Přátelští ale i nepřátelští agenti po zničení budou odhazovat různé předměty, které lze sebrat a hráči dokáží nějak pomoci. Hráč bude mít přístup k "náhlavnímu displeji"(HUD) na obrazovce, kde bude moci sledovat důležité informace o sobě, ale HUD také usnadní navigaci v hlavním menu a menu pauzy.

3.2 Algoritmy pro vytvoření agenta v akční hře

Oba algoritmy jsou založené na Q-Learning algoritmu.

Q-Learning

Q-Learning[38] je populární algoritmus RL, který se používá k určení optimální politiky výběru akce pro agenta, který reaguje na prostředí. Pracuje v diskrétních časových krocích a jeho cílem je naučit se optimální politiku iterativní aktualizací hodnot Q , které představují očekávanou užitečnost provedení určité akce v daném stavu.

V Q-Learning si agent vede Q -tabulku, kde každá položka $Q(s, a)$ představuje odhadovanou hodnotu přijetí akce a ve stavu s . Agent aktualizuje tyto hodnoty Q na základě získaných odměn a odhadovaných budoucích odměn. Konkrétně po provedení akce a pozorování výsledné odměny a nového stavu se hodnota Q aktualizuje pomocí Bellmanovy rovnice:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (3.1)$$

Zde α je míra učení, γ je diskontní faktor, r je získaná odměna a s' je nový stav. Výraz $\max_{a'} Q(s', a')$ představuje maximální odhadovanou budoucí odměnu za nový stav. Tímto procesem se Q-Learning snaží konvergovat k optimálním hodnotám Q , z nichž lze odvodit optimální politiku výběrem akcí s nejvyššími hodnotami Q pro každý stav.

3.2.1 SARSA (State-Action-Reward-State-Action)

SARSA(Stav-Akce-Odměna-Stav-Akce)[36] je RL algoritmus na základě politiky, který se používá k nalezení optimální politiky pro agenta, který interaguje s prostředím. Název „SARSA“ je odvozen od posloupnosti událostí, které používá k aktualizaci hodnot Q : aktuální stav s , akce provedená v tomto stavu a , odměna získaná po provedení akce s , další stav s' a další akce provedená v novém stavu a' .

Jádrem metody SARSA je odhad funkce **Q-hodnoty**, $Q(s, a)$, která představuje očekávanou kumulativní odměnu za provedení akce a ve stavu s a následné dodržování určité politiky. Na rozdíl od jiných algoritmů učení s posilováním, jako je Q-Learning, který je off-policy, je SARSA algoritmus on-policy. To znamená, že aktualizuje své hodnoty Q na základě akcí skutečně provedených agentem podle jeho aktuální politiky, včetně akcí průzkumu.

Díky tomu je SARSA obzvláště užitečná v prostředích, kde je politika agenta nedeterministická nebo kde agent potřebuje vyvážit průzkum (zkoušení nových akcí) a využití (výběr nejlepší známé akce).

Pravidlo aktualizace pro SARSA zní:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)] \quad (3.2)$$

kde:

- $Q(s, a)$ je aktuální odhad hodnoty Q pro dvojici stav-akce (s, a) ,
- α je míra učení, která určuje, jak moc nové informace převáží nad starými,
- r je odměna získaná po provedení akce a ,
- γ je diskontní faktor, který určuje důležitost budoucích odměn,
- $Q(s', a')$ je hodnota Q příští dvojice stav-akce.

Algoritmus **SARSA** funguje následovně:

1. **Inicializace:** Inicializujte funkci Q -hodnoty $Q(s, a)$ libovolně pro všechny dvojice stav-akce (s, a) . Inicializujte míru učení α , diskontní faktor γ a politiku průzkumu ϵ .
2. **Zahájení epizody:** Pozoruje počáteční stav s a zvolí akci a na základě aktuální politiky (např. $\epsilon - greedy$).
3. **Provedení akce:** Proveďte akci a , sleduje odměnu r a další stav s' .
4. **Výběr další akce:** Vybere další akci a' ve stavu s' pomocí aktuální politiky.
5. **Aktualizace hodnoty Q :** Aktualizuje hodnotu Q pro aktuální dvojici stav-akce (s, a) pomocí pravidla 3.2
6. **Přechod:** Nastaví $s \leftarrow s'$ a $a \leftarrow a'$. Opakuje kroky 3 až 5, dokud epizoda neskončí.
7. **Konec epizody:** Pokud epizoda skončí, spustí novou epizodu a postup opakuje až do konvergence.

SARSA obvykle používá $\epsilon - greedy$ politiku, kde agent vybírá akci, o které se domnívá, že má nejvyšší hodnotu Q s pravděpodobností $1 - \epsilon$ a zkoumá náhodné akce s pravděpodobností ϵ . Tato rovnováha umožňuje agentovi dostatečně prozkoumat prostředí a zároveň využít znalosti, které nashromáždil.

SARSA se nejčastěji používá na automatické hraní her nebo v robotice může pomoci při výcviku robotů, aby se lépe orientovali v prostředí, kde nemusí být důsledky akcí okamžitě zřejmé.

3.2.2 Deep Q-Learning

Deep Q-Learning (DQN)[39][40] je rozšířením tradičního algoritmu Q-Learning, které je speciálně navrženo pro práci s komplexními prostředím s vysokodimenzionálními stavovými prostory, jaké se vyskytují například ve videohrách nebo v robotice. V klasickém Q-Learningu se k ukládání hodnot dvojic stav-akce používá Q-tabulka. Tento přístup se však stává nepraktickým, pokud se jedná o prostředí, kde je stavový prostor příliš velký na to, aby jej bylo možné explicitně reprezentovat. DQN toto omezení překonává využitím hlubokých neuronových sítí k aproximaci funkce Q, která mapuje stavy na hodnoty Q pro každou možnou akci. Takže namísto udržování velké tabulky hodnot Q se použije hluboká neuronová síť k předpovídání hodnot Q pro každou akci danou konkrétním stavem. Tato neuronová síť, často označovaná jako Q-síť, přijímá jako vstup aktuální stav prostředí a na výstupu má vektor Q-hodnot, jeden pro každou možnou akci. Akce s nejvyšší hodnotou Q je pak vybrána v souladu se zásadami.

Trénování sítě Q spočívá v minimalizaci rozdílu mezi předpovídanou hodnotou Q a cílovou hodnotou Q, která se vypočítá pomocí Bellmanovy rovnice. Cílová hodnota Q se vypočítá jako součet okamžité odměny a diskontované maximální budoucí odměny, kterou odhaduje samotná Q-síť. To přináší problém nestability během trénování, protože cílová hodnota závisí také na vahách sítě. Ke zmírnění tohoto problému využívá metoda Deep Q-Learning dvě klíčové techniky: přehrávání minulosti a cílová síť.

Přehrávání minulosti je metoda, při níž je minulost agenta, reprezentovaná jako tuply stavu, akce, odměny a dalšího stavu, ukládána do vyrovnávací paměti pro znovupoužití. Během tréninku se z této vyrovnávací paměti náhodně vybírají tuply, aby se přerušila korelace mezi po sobě jdoucími stavy, a tím se stabilizovalo učení. To umožňuje síti učit se z rozmanitějšího souboru, což zlepšuje celkovou robustnost politiky.

Cílová síť je druhá síť DQN, která má identickou architekturu jako síť Q, ale její parametry se aktualizují méně často. Tato síť se používá k vytváření cílových hodnot Q, což zajišťuje, že aktualizace směřují ke stabilnějšímu cíli. Váhy cílové sítě jsou pravidelně aktualizovány kopírováním vah Q-sítě, což pomáhá dále stabilizovat proces učení.

Kroky algoritmu DQN:

1. Inicializace:

- Inicializace **Q-sítě** s náhodnými váhami. Tato neuronová síť bude aproximovat Q-funkci, která předpovídá hodnotu každé akce na základě stavu.
- Inicializace **cílové sítě** se stejnou architekturou a váhami jako Q-síť. Cílová síť bude použita k výpočtu stabilních cílových Q-hodnot.
- Nastavení **přehrávacího bufferu** pro uchovávání minulosti agenta. Tento buffer pomůže přerušit korelaci mezi po sobě jdoucími stavy a poskytne různorodý soubor tréninkových dat.

2. Interakce s prostředím:

- Pro každou epizodu:
 - **Resetujte prostředí** na počáteční stav a inicializujte epizodu.
 - Pro každý časový krok v epizodě:
 - * **Vyberte akci:** Vyberte akci a pomocí $\epsilon - greedy$ politiky. S pravděpodobností $1 - \epsilon$ vyberte akci s nejvyšší Q-hodnotou, jak ji předpovídá Q-síť. S pravděpodobností ϵ vyberte náhodnou akci pro zajištění explorační.
 - * **Proveďte akci:** Proveďte zvolenou akci a v prostředí a pozorujte získanou odměnu r a následující stav s' .
 - * **Uložte zkušenost:** Přidejte dvojici (s, a, r, s') do přehrávacího bufferu.
 - * **Vytvořte mini-batch:** Náhodně vyberte mini-batch zkušeností z přehrávacího bufferu.
 - * **Aktualizujte Q-síť:** Pro každou zkušenost v mini-batch:
 - Vypočítejte **cílovou Q-hodnotu** pomocí cílové sítě:

$$y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$$

kde γ je diskontní faktor a $\max_{a'} Q_{\text{target}}(s', a')$ je maximální Q-hodnota pro následující stav s' podle cílové sítě.

- Aktualizujte Q-síť minimalizováním **průměrné kvadratické chyby** mezi předpovězenou Q-hodnotou a cílovou Q-hodnotou:

$$\text{Loss} = (Q(s, a) - y)^2$$

- * **Periodická aktualizace:** Periodicky aktualizujte váhy cílové sítě zkopírováním vah z Q-sítě.

3. Opakování:

- Pokračujte v uvedených krocích pro každou epizodu, dokud proces učení nezkonverguje nebo nedosáhne kritéria zastavení.

4. Hodnocení:

- Hodnoťte naučenou politiku testováním agenta v prostředí, sledujte jeho výkon a podle potřeby upravte parametry.

3.2.3 Rozdíly mezi těmito algoritmy

SARSA vs DQN:

- **On-Policy vs Off-Policy:** SARSA aktualizuje Q-hodnoty na základě akcí prováděných současnou politikou, což vede k stabilnímu učení. DQN naopak používá přehrávání minulosti a cílové sítě pro stabilní a efektivní učení.
- **Jednoduchost:** SARSA má snadnou implementaci a pochopení, vhodné pro malé prostory stavů a akcí. DQN naopak je složitější na implementaci a ladění.
- **Explorace:** SARSA konzervativně exploruje a tím snižuje riziko nesprávného učení, ale má problém s vyvážením explorační a využití. DQN zase lépe řídí vyvážení mezi explorační a využití, ale může se špatně naučit.
- **Tabulární omezení:** SARSA je nevhodné pro velké nebo kontinuální prostory stavů. DQN naopak dobře zvládá velké a kontinuální prostory stavů díky hlubokým neuronovým sítím.
- **Potenciální suboptimálnost:** SARSA může konvergovat k suboptimálním politikám, pokud není explorační dobře řízena. DQN zase může trpět nestabilitou, i když pokročilé techniky mohou toto zmírnit.
- **Požadavky na výpočetní výkon:** DQN potřebuje značné výpočetní zdroje a čas. SARSA díky jednoduchosti je nenáročná.

3.2.4 Použití obou algoritmů pro vytvoření agenta

DQN je obecně lépe přizpůsoben pro složité herní AI s velkými prostory stavů a různorodými scénáři díky jeho schopnosti efektivně řídit explorační a využití a navíc zvládat složité prostředí prostřednictvím hlubokých neuronových sítí, ale SARSA stále může být použit v jednodušších nebo více kontrolovaných prostředích, kde jsou prostory stavů a akcí zvládnutelné, a kde jsou přednostmi stabilita a jednoduchost implementace před zvládáním komplexity.

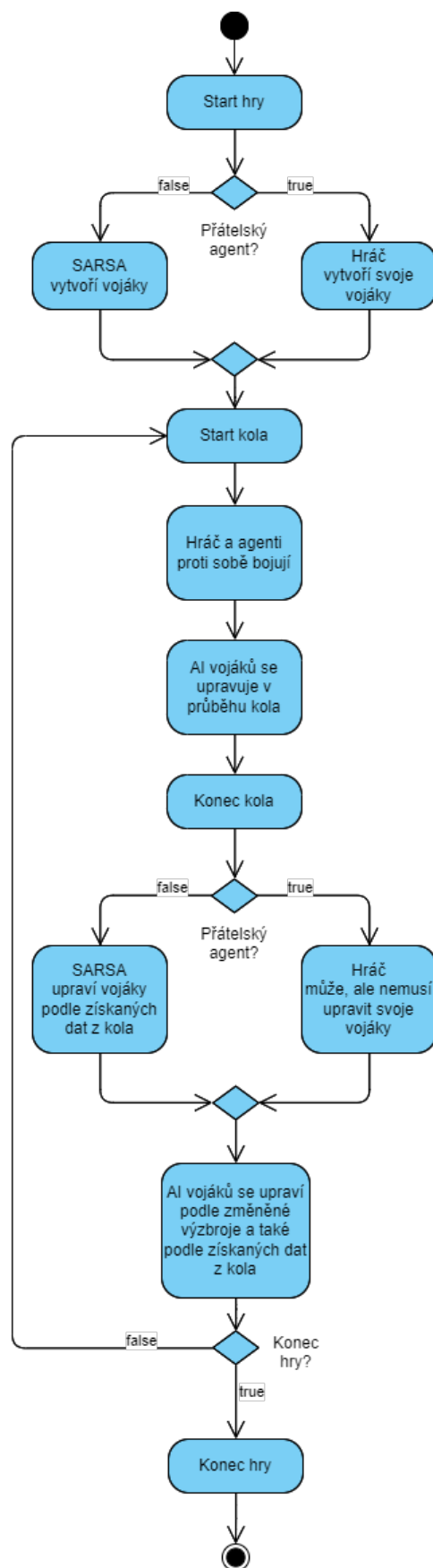
Díky těmto znalostem dostanou oba algoritmy tohoto agenta vlastní úkol. Agent se bude skládat z logiky na oblékání svých jednotek, kde se využije SARSA, protože je to velice kontrolované prostředí, kde se nemůže neustále náhodně měnit, bude mít algoritmus pouze předem určené množství různého vybavení, podle kterého bude moct sestavit voják. Pro samotné AI, které se bude rozhodovat na bitevním poli, se použije Deep Q-Learning právě kvůli složitosti prostředí a neustále se měnícím podmínkám, které budou často mezi sebou neporovnatelné hlavně i díky hráčovi, který se bude zapojovat do bitev, protože hra bude z pohledu první osoby a nebude pouze ovládat svoje vojáky. Hráčovo vojáků nebudou obsahovat SARSA část logiky, protože hráč si bude sestavovat vojáků sám, ale stále jejich hlavní logika bude fungovat jako u nepřátelského AI pomocí Deep Q-Learning.

Kapitola 4

Návrh hodnocení agenta

4.1 Metodika hodnocení

Hra bude rozdělena na kola, kde jedna hra se bude skládat z neurčitých počet kol. Hra je navržena tak, že bojuje hráč se svými vojáky proti nepřátelským vojákům, kteří jsou čistě řízeni autonomním agentem. Když hra začne, tak hráč si postaví vlastní vojáky, kteří mu budou pomáhat bojovat a zvítězit proti nepřátelským vojákům, kteří budou vytvořeni podle předem naučeného SARSA algoritmu. V průběhu kola se také předem naučená AI část vojáků, která by byla vytvořena za pomoci Deep Q-Learning algoritmu, bude v průběhu bitvy upravovat, aby se pokusili nepřátelští vojáci přelstít hráče a jeho vojáky a díky tomu je porazit. Na konci kola sesbírané informace hráč využije k tomu, zda chce nebo potřebuje upravit svoje vojáky na další kolo. Agent využije svoje nasbírané informace k úpravě SARSA algoritmu, aby dokázal vybrat efektivnější výbavu proti hráči. Nakonec tyto informace se pošlou do Deep Q-Learning algoritmu s informacemi z kola a upraví se tím chování vojáků, aby dokázali lépe a efektivněji porazit hráče a jeho vojáky. Pokud toto není konec hry, tak s těmito novými úpravami a novou výbavou začne další kolo. Tento cyklus se opakuje, dokud nenastane konec hry. Když konec hry nastane, tak se algoritmy revertují na svůj základní předem naučený stav.



Obrázek 4.1: Diagram logiky hry, která byla popsána výše.

4.2 Typy hodnocení

Agenta lze hodnotit řadou způsobů, ale zvoleny byly tyto tři návrhy, protože budou mít největší asi i nejzajímavější efekt na chování agenta umělé inteligence:

4.2.1 Podle efektivnosti zabití nepřátel

Když se agent bude hodnotit podle množství nepřátel, které dokáže porazit, agent si pravděpodobně přizpůsobí chování tak, že bude neustále útočit a bude mít agenty přizpůsobené pravděpodobně na rychlost a jak rychle dokáže nepřítele porazit, i když by agent měl velice malé množství života a bude např. vidět hůře atd. Agent se pravděpodobně bude i málo schovávat, protože by mu schovávání snížilo rychlost a množství poražených nepřátel. Díky těmto všem změnám by i mohl mít pravděpodobně možnost posílat více jednotek, což stíží boj hráčovi, protože se bude muset soustředit na vícero nepřátel zároveň. Nebo agent může zkusit jiný směr a vytvoří jednu super jednotku, která by měla hodně života, brnění a používala by zbraň, co uděluje velké poškození, ale nemohl by se skoro pohybovat a viděl by ještě hůře. Tato jednotka by byla velice náročná na zničení, ale když by se jí podařilo zničit, nemá nepřítel možnost vyslat jednotky další.

4.2.2 Podle efektivnosti přežití

Agent se bude nejpravděpodobněji chovat úplně naopak, když se bude hodnotit podle toho, jak dlouho dokáže jedna jednotka přežít, nežli kolik dokáže jednotek nepřítele porazit. Agent by se soustředil na množství života a brnění a nezajímalo by ho, kolik dokáže udělit poškození hráčovi a nepřátelským agentům. Tohle je jeden ze směrů, kterými by se mohl vydat, nebo by taky mohl zkusit vytvořit jednotku, která se pohybuje rychle a útočí z dálky pomocí např. odstřelovací pušky, místo toho, aby bojoval v blízkosti ostatních nepřátel. Agent nejpravděpodobněji se nebude snažit používat ani malé, ale ani velké množství jednotek, aby mohl zefektivnit přežití ostatních.

4.2.3 Efektivita zabití + přežití

Nejzajímavější situace je ta, kde by se hodnotilo i přežití, ale zároveň i efektivita zničení nepřítele. V této situaci je dost možné mít velké množství různých scénářů, ale zároveň tento scénář může být ten nejméně zajímavý, protože je zde vysoká šance, že si agent najde scénář, který je perfektní vůči tomuto hodnocení mnohem rychleji, než v obou předchozích scénářích. Např. by agent mohl vytvořit „Super odstřelovače“, který by měl celkem dobré množství života, brnění a rychlosti a zároveň by utíkal od nepřátel a snažil by se je z dálky ničit pomocí odstřelovací pušky. I když by byl úplně sám, pokud nemá tým hráče jednotky, co dokážou tuto jednotku z této dálky zneškodnit, byla by to velmi rychlá prohra pro hráče.

Závěr

Hlavním cílem tohoto výzkumného úkolu bylo navrhnout agenta umělé inteligence pro 3D akční hry za pomoci algoritmů strojového učení, díky kterým by se AI dokázalo plně přizpůsobovat jakýmkoliv situacím, ostatnímu AI nebo také hráči.

Nejdříve jsem popsal základní historii umělé inteligence ve hrách a jaké techniky/algoritmy se k tomu nejčastěji používají. Ještě jsem se zaměřil na AI ve 3D akčních hrách a specificky na hry v tomto žánru, které mají mnohem inteligentnější umělou inteligenci oproti průměrné hře.

V další části jsem vypsál zjednodušené základy strojového učení, které vedou k popisu všech typů strojového učení, kde u každého učení byli také vylíčeny některé z algoritmů pro daný typ strojového učení.

Ve třetím úseku jsem popsal návrh samotného autonomního agenta, kde popisují typ vizualizace, který bych použil a samotný popis i prostředí, kde by se agent využil. Také tu popisují a porovnávám dva algoritmy, které by se použily ke sestrojení dvou odlišných částí agenta.

V posledním segmentu jsem vysvětlil metodiku a principy samotného hodnocení agenta, pomocí kterého by se upravovaly vnitřní hodnoty obou algoritmů popsaných v předchozí části.

Tento návrh agenta a prostředí by byl realizován v diplomové práci, kde by byla vysvětlena i samotná konstrukce a jestli je tato domněnka vůbec možná sestavit.

Literatura

- [1] SAMUEL, Arthur Lee. Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*. 1959, roč. 3, č. 3, s. 210-229. DOI 10.1147/rd.33.0210.
- [2] SCHAEFFER, Jonathan; BURCH, Neil; BJÖRNSSON, Yngvi; KISHIMOTO, Akihiro; MÜLLER, Martin et al. Checkers Is Solved. *Science*. 2007, roč. 317, č. 5844, s. 1518-1522. DOI 10.1126/science.1144079.
- [3] Wikipedia contributors. *TD-Gammon*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: <https://en.wikipedia.org/w/index.php?title=TD-Gammon&oldid=1208876151>. [cit. 2024-02-07].
- [4] Wikipedia contributors. *Deep Blue*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: [https://en.wikipedia.org/w/index.php?title=Deep_Blue_\(chess_computer\)&oldid=1208692792](https://en.wikipedia.org/w/index.php?title=Deep_Blue_(chess_computer)&oldid=1208692792). [cit. 2024-02-07].
- [5] GOOGLE. *AlphaGo*. Online. GOOGLE. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/technologies/alphago/>. [cit. 2024-02-07].
- [6] Oppy, Graham and Dowe a David. *The Turing Test*. Online. Stanford Encyclopedia of Philosophy. Winter 2021. Dostupné z: <https://plato.stanford.edu/archives/win2021/entries/turing-test/>. [cit. 2024-02-08].
- [7] GILL, Arthur. *Introduction to the Theory of Finite-State Machines*. 1. McGraw Hill, 207n. 1. ISBN 9780070232433.
- [8] COLLEDANCHISE, Michele a OGREN, Petter. *Behavior Trees in Robotics and AI: An Introduction*. Online. CRC Press, 2018. ISBN 9781138593732. Dostupné z: <https://doi.org/10.1201/9780429489105>. [cit. 2024-02-26].
- [9] ISLA, Damian. *Handling Complexity in the Halo 2 AI*. Online. In: Game Developer. C2024. Dostupné z: <https://www.gamedeveloper.com/programming/gdc-2005-proceeding-handling-complexity-in-the-i-halo-2-i-ai>. [cit. 2024-02-10].
- [10] GRAHAM, David „Rez“ a RABIN, Steve. An Introduction to Utility Theory. Online. In: *Game AI Pro 360: Guide to Architecture*. CRC Press, 2019, s. 67-80. ISBN 9780429055058. Dostupné z: <https://doi.org/10.1002/9780429055058.ch4>. [cit. 2024-02-07].

- [//www.taylorfrancis.com/chapters/edit/10.1201/9780429055058-6/introduction-utility-theory-david-rez-graham](https://www.taylorfrancis.com/chapters/edit/10.1201/9780429055058-6/introduction-utility-theory-david-rez-graham). [cit. 2024-02-10].
- [11] DILL, Kevin a MARK, Dave. *Improving AI Decision Modeling Through Utility Theory*. Online. In: GDCVault. 1986. Dostupné z: <https://www.gdcvault.com/play/1012410/Improving-AI-Decision-Modeling-Through>. [cit. 2024-02-10].
 - [12] Wikipedia contributors. *Tree traversal*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=Tree_traversal&oldid=1205867095. [cit. 2024-02-10].
 - [13] YU, Xinjie a GEN, Mitsuo. *Introduction to Evolutionary Algorithms*. Online. Springer London, 2010. ISBN 978-1-84996-129-5. Dostupné z: <https://doi.org/https://doi.org/10.1007/978-1-84996-129-5>. [cit. 2024-02-10].
 - [14] GAMESPOT STAFF. *F.E.A.R. Designer Diary #1 - A Study of Smart AI, Part I*. Online. GameSpot. C2024. Dostupné z: <https://www.gamespot.com/articles/fear-designer-diary-1-a-study-of-smart-ai-part-i/1100-6133519/>. [cit. 2024-02-11].
 - [15] GAMESPOT STAFF. *F.E.A.R. Designer Diary #2 - A Study of Smart AI, Part II*. Online. GameSpot. C2024. Dostupné z: <https://www.gamespot.com/articles/fear-designer-diary-2-a-study-of-smart-ai-part-ii/1100-6134022/>. [cit. 2024-02-11].
 - [16] FIKES, Richard E. a NILSSON, Nils J. Strips: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*. Winter 1971, roč. 2, č. 3-4, s. 189-208. ISSN 0004-3702.
 - [17] ORKIN, Jeff. *Three States and a Plan: The A.I. of F.E.A.R.* Online. In: Semantic Scholar. 2015. Dostupné z: <https://api.semanticscholar.org/CorpusID:62493110>. [cit. 2024-02-11].
 - [18] PARKIN, Jeffrey. *Middle-earth: Shadow of War guide: The Nemesis System*. Online. In: Polygon. C2024. Dostupné z: <https://www.polygon.com/middle-earth-shadow-of-war-guide/2017/10/9/16439610/the-nemesis-system-and-you>. [cit. 2024-03-12].
 - [19] Game Maker's Toolkit. *How the Nemesis System Creates Stories*. Online. In: Youtube. C2005-2024. Dostupné z: https://www.youtube.com/watch?v=Lm_AzK27mZY. [cit. 2024-03-12].
 - [20] RUSSELL, Stuart J. a NORVIG, Peter. *Artificial Intelligence : A Modern Approach*. Englewood Cliffs: Prentice Hall, c1995. ISBN 9780131038059.
 - [21] GHORBANI, Fardin; SHABANPOUR, Javad; BEYRAGHI, Sina; SOLEIMANI, Hossein; ORAIZI, Homayoon et al. A deep learning approach for inverse design of the metasurface for dual-polarized waves. Online. *Applied Physics A*. 2021, roč. 127, č. 869, s. 3. ISSN 1432-0630. Dostupné z: <https://doi.org/10.1007/s00339-021-05030-6>. [cit. 2024-04-09].

- [22] STEINWART, Ingo a CHRISTMANN, Andreas. *Support Vector Machines*. Information Science and Statistics. New York, NY: Springer New York, 2008. ISBN 978-0-387-77242-4.
- [23] CHEN, Nongtian; SUN, Youchao; WANG, Zongpeng a PENG, Chong. Improved LS-SVM Method for Flight Data Fitting of Civil Aircraft Flying at High Plateau. Online. *Electronics*. 2022, roč. 11, č. 10, s. 12. ISSN 2079-9292. Dostupné z: <https://doi.org/10.3390/electronics11101558>. [cit. 2024-05-08].
- [24] BREIMAN, Leo; FRIEDMAN, Jerome; OLSHEN, R.A. a STONE, Charles J. *Classification and Regression Trees*. Online. New York: Chapman and Hall/CRC, 2017. ISBN 9781315139470. Dostupné z: <https://doi.org/https://doi.org/10.1201/9781315139470>. [cit. 2024-05-08].
- [25] BOHRA, Aman. *CLASSIFICATION BY DECISION TREE INDUCTION*. Online. In: SPPU BE Computer Laboratory(I,II,III,IV) And Project RELATED STUDY MATERIAL. 2016. Dostupné z: <https://becomputerlaboratory.blogspot.com/2016/10/classification-by-decision-tree.html?m=1>. [cit. 2024-05-08].
- [26] ENGELBRECHT, Andries P. Unsupervised Learning Neural Networks. *Computational Intelligence: An Introduction*. 2007, roč. 2, č. 4, s. 55-7. DOI 10.1002/9780470512517.
- [27] ROHIT, M. *Introduction to Unsupervised Learning*. Online. In: Bombay Softwares. 2023. Dostupné z: <https://www.bombaysoftwares.com/blog/introduction-to-unsupervised-learning>. [cit. 2024-08-12]
- [28] MACQUEEN, J. Some methods for classification and analysis of multivariate observations. Online. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*. 1967, roč. 5, č. 1, s. 281-297. Dostupné z: <https://projecteuclid.org/ebooks/berkeley-symposium-on-mathematical-statistics-and-probability/Proceedings-of-the-Fifth-Berkeley-Symposium-on-Mathematical-Statistics-and-probability/chapter/Some-methods-for-classification-and-analysis-of-multivariate-observations/bsmsp/1200512992?tab=ChapterArticleLink>. [cit. 2024-08-12].
- [29] KAUFMAN, Leonard a ROUSSEEUW, Peter. *Finding Groups in Data: An Introduction To Cluster Analysis*. Wiley, New York, 1990. ISBN 0-471-87876-6. DOI 10.2307/2532178. [cit. 2024-08-12].
- [30] Wikipedia contributors. *Dendrogram*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: <https://en.wikipedia.org/w/index.php?title=Dendrogram&oldid=1195077958>. [cit. 2024-08-012].
- [31] AGRAWAL, Rakesh a IMIELIŃSKI, Tomasz a SWAMI, Arun. Mining association rules between sets of items in large databases. Online. *SIGMOD Rec.* 1993, roč. 22, č. 2, s. 207-216. Dostupné z: <https://doi.org/10.1145/170036.170072>. [cit. 2024-08-12]

- [32] AGRAWAL, Rakesh a SRIKANT, Ramakrishnan. Fast Algorithms for Mining Association Rules in Large Databases. *Proceedings of the 20th International Conference on Very Large Data Bases*. 1994, roč. 20, s. 487-499. DOI 10.5555/645920.672836. [cit. 2024-08-12]
- [33] SRIKANT, Ramakrishnan a AGRAWAL, Rakesh. Mining sequential patterns: Generalizations and performance improvements. *Advances in Database Technology — EDBT '96*. 1996, roč. 1057, s. 1-17. Dostupné z: <https://doi.org/10.1007/BFb0014140>. [cit. 2024-08-12]
- [34] GOODFELLOW, Ian; POUGET-ABADIE, Jean; MIRZA, Mehdi; XU, Bing; WARDE-FARLEY, David et al. Generative Adversarial Nets. Online. *Advances in Neural Information Processing Systems*. 2014, roč. 27, s.9. Dostupné z: https://papers.nips.cc/paper_files/paper/2014/hash/5ca3e9b122f61f8f06494c97b1afccf3-Abstract.html. [cit. 2024-08-12].
- [35] GOOGLE. *Overview of GAN Structure*. Online. In: Machine Learning. 2022. Dostupné z: <https://developers.google.com/machine-learning/gan/gan-structure>. [cit. 2024-08-12]
- [36] SUTTON, Richard S. a Andrew G, BARTO. *Reinforcement Learning: An Introduction*. Online. Bradford Book, 1998. ISBN 0262039249. [cit. 2024-05-08].
- [37] ROUSSO, Pat; SAMSAM, Sanaz a CHHABRA, Robin. *A Mission Architecture for On-Orbit Servicing Industrialization*. 2021, s. 1-14. DOI 10.1109/AERO50100.2021.9438404. [cit. 2024-08-12]
- [38] WATKINS, Christopher J. C. H.; DAYAN, Peter. Q-learning. *Machine Learning*. 1992, roč. 8, s. 279–292. Dostupné z: <https://doi.org/10.1007/BF00992698>. [cit. 2024-08-12]
- [39] NUER, Jeremi. *A Comprehensive Guide to Deep Q-Learning*. Online. In: Medium. 2022. Dostupné z: <https://medium.com/@jereminuerofficial/a-comprehensive-guide-to-deep-q-learning-8aeed632f52f>. [cit. 2024-08-12].
- [40] *Deep Q-Learning*. Online. In: GeeksforGeeks. 2023. Dostupné z: <https://www.geeksforgeeks.org/deep-q-learning/>. [cit. 2024-08-12].